



Diagnóstico de defeitos de impressão em etiquetas de códigos de barras: uma abordagem por visão computacional e orquestração multiagente

Diagnosis of printing defects in barcode labels: an approach based on computer vision and multi-agent orchestration

Saymon Coppi de Oliveira Silva*

2026

Resumo

A leitura confiável de códigos de barras depende da qualidade de impressão das etiquetas, avaliada por normas como a ISO/IEC 15416; contudo, os verificadores certificados são caros e reportam apenas uma nota, sem indicar a causa física do defeito nem a ação corretiva. Este trabalho propõe e desenvolve um sistema de baixo custo que, a partir de uma imagem capturada por um dispositivo comum, detecta defeitos típicos de impressão em etiquetas de códigos de barras e sugere sua causa provável. A solução combina visão computacional — uma rede neural convolucional com *backbone* leve, treinada por transferência de aprendizado — e orquestração multiagente com o *Agent Development Kit* (ADK), decompondo a tarefa em leitura/decodificação, estimativa de indicadores de qualidade, detecção de defeitos físicos e diagnóstico de causa-raiz fundamentado na documentação técnica do fabricante — estágio que, além dos resultados anteriores, inspeciona a própria imagem (multimodal) e pode arbitrar a classificação da rede nos pares visualmente confundíveis. O sistema é exposto por uma *API* única, consumida por uma interface web responsiva, acessível de qualquer dispositivo com navegador — computador, *notebook*, *tablet* ou celular —, independentemente do sistema

*Universidade Federal de São Paulo (Unifesp), Instituto de Ciência e Tecnologia. Programa de Pós-Graduação em Ciência da Computação. Disciplina: Visão Computacional (cód. 2587), Prof. Dr. Fabio Augusto Menocci Cappabianco. E-mail: saymoncoppi@gmail.com.

operacional. A avaliação emprega um conjunto de dados híbrido — imagens reais de captura e defeitos de impressão gerados sinteticamente de forma controlada e reproduzível — e, para a classificação de defeitos, compara a CNN adotada (MobileNetV3-Small) a arquiteturas de referência (ResNet18 e EfficientNet-B0) sob protocolo idêntico, com validação cruzada estratificada e teste de significância estatística. Os resultados demonstram a viabilidade de um apoio à decisão que vai além da nota, indicando causa provável e correção sugerida.

Palavras-chave: código de barras; qualidade de impressão; visão computacional; aprendizado profundo; sistemas multiagentes.

Abstract

Reliable barcode reading depends on the print quality of the labels, which is assessed by standards such as ISO/IEC 15416; however, certified verifiers are expensive and report only a grade, without indicating the physical cause of the defect or the corrective action. This work proposes and develops a low-cost system that, from an image captured by an ordinary device, detects typical printing defects in barcode labels and suggests their probable cause. The solution combines computer vision — a convolutional neural network with a lightweight backbone, trained by transfer learning — and multi-agent orchestration with the Agent Development Kit (ADK), decomposing the task into reading/decoding, quality-indicator estimation, physical-defect detection and root-cause diagnosis grounded in the manufacturer's technical documentation — a stage that, beyond the previous results, also inspects the image itself (multimodal) and can arbitrate the network's classification for visually confusable pairs. The system is exposed through a single API, consumed by a responsive web interface accessible from any device with a browser — desktop, notebook, tablet or smartphone —, regardless of the operating system. The evaluation uses a hybrid dataset — real capture images and printing defects generated synthetically in a controlled and reproducible way — and, for defect classification, compares the adopted CNN (MobileNetV3-Small) against reference architectures (ResNet18 and EfficientNet-B0) under an identical protocol, with stratified cross-validation and a statistical significance test. The results demonstrate the feasibility of a decision-support tool that goes beyond the grade, pointing to a probable cause and a suggested correction.

Keywords: barcode; print quality; computer vision; deep learning; multi-agent systems.

Sumário

1	Introdução	4
1.1	Contextualização	4
1.2	Problema de pesquisa	4
1.3	Objetivos	5
1.4	Contribuições do trabalho	5
1.5	Organização do artigo	6
2	Fundamentação teórica	6
2.1	Códigos de barras e simbologias	6
2.2	Qualidade de impressão e verificação (ISO/IEC 15416 e 15415)	8
2.3	Defeitos típicos de impressão térmica e suas causas	9

2.4	Processamento de imagens e visão computacional para inspeção	9
2.5	Redes convolucionais, MobileNetV3 e transferência de aprendizado	10
2.6	Sistemas multiagentes e o <i>Agent Development Kit</i> (ADK)	12
3	Trabalhos relacionados	12
3.1	Verificação normativa e diagnóstico de defeitos	12
3.2	Inspeção visual de defeitos com aprendizado profundo	13
3.3	Sistemas multiagentes aplicados a tarefas de visão	13
3.4	Lacuna que este trabalho preenche	13
3.5	Comparação explícita com <i>Research on a Multi-Type Barcode Defect De- tection Model Based on Machine Vision</i>	14
4	Metodologia	16
4.1	Tipo de pesquisa	16
4.2	Conjunto de dados	17
4.3	Ferramentas utilizadas	18
4.4	Ambiente experimental	19
4.5	Métricas de avaliação	20
5	Proposta da solução	20
5.1	Arquitetura da solução	20
5.2	Fluxograma do processamento	21
5.3	Algoritmos empregados	22
5.4	Detalhes da implementação	23
5.5	Interface de uso: chat de inspeção	23
6	Resultados e discussão	24
6.1	Configuração dos experimentos	24
6.2	Resultados quantitativos	24
6.3	Comparação com <i>baselines</i> e validação cruzada	25
6.4	Multiagente <i>versus</i> abordagem monolítica	26
6.5	Progressão empírica e a fronteira confundível: escala de dado real	27
6.6	Análise e discussão	28
6.7	Limitações	29
7	Conclusão	29

Referências Bibliográficas	30
---	-----------

1 Introdução

1.1 Contextualização

Códigos de barras são a espinha dorsal da identificação automática de itens em varejo, logística, saúde e manufatura. Uma etiqueta é lida dezenas de vezes ao longo da cadeia — na expedição, no transporte, no recebimento e no ponto de venda — e cada leitura pressupõe que a impressão preserve as proporções de barras e espaços dentro de tolerâncias estreitas. Quando a impressão degrada, o custo não é apenas uma releitura: há reproprocessamento manual, atraso operacional, devolução de cargas e, em setores regulados, não conformidade.

Por essa criticidade, a qualidade de impressão é normatizada. A norma ISO/IEC 15416 define, para símbolos lineares (1D), um conjunto de parâmetros objetivos derivados do *perfil de refletância de varredura* — entre eles contraste do símbolo, modulação, decodificabilidade e defeitos — que são convertidos em notas de A a F (ISO/IEC, 2016). Para símbolos bidimensionais (2D), a ISO/IEC 15415 cumpre papel análogo (ISO/IEC, 2024), e a conformidade dos aparelhos verificadores é regida pela ISO/IEC 15426 (ISO/IEC, 2006). Na prática industrial, essa avaliação é feita por *verificadores ópticos* dedicados, equipamentos calibrados e de custo elevado.

Paralelamente, no chão de fábrica, os defeitos de impressão têm origem física bem conhecida e documentada pelos fabricantes. No caso da Zebra Technologies — referência deste trabalho —, os guias das impressoras industriais da série ZT (ZT411/ZT421) e a base de conhecimento associam sintomas visuais a causas mecânicas concretas: nível de *darkness* (temperatura) incorreto, pressão desigual da cabeça de impressão, enrugamento ou rompimento do *ribbon*, cabeça de impressão suja ou com elemento queimado, rolete (*platen*) desgastado ou sujo, e incompatibilidade entre mídia e *ribbon* (Zebra Technologies, 2024d). Ou seja, existe um corpo de conhecimento que liga aparência do defeito, componente responsável e ação corretiva.

1.2 Problema de pesquisa

Há uma lacuna entre esses dois mundos. Os verificadores certificados são caros, pouco portáteis e, sobretudo, reportam uma nota, não a causa-raiz: informam que o símbolo é grau “C” ou “D”, mas não dizem ao operador *por que* — se é *ribbon* enrugado, pressão desigual ou cabeça queimada — nem *o que fazer*. Do outro lado, o conhecimento de causa-raiz existe, mas está disperso em manuais e depende da experiência de um técnico para ser aplicado.

Este trabalho investiga a seguinte questão de pesquisa:

É possível, a partir de uma imagem capturada por um dispositivo comum, identificar defeitos típicos de impressão de códigos de barras e sugerir sua causa provável, combinando visão computacional e um sistema multiagente orquestrado, com precisão suficiente para apoiar a decisão de um operador?

Escopo do diagnóstico. O sistema **não é um verificador certificado** e não substitui a medição conforme ISO/IEC 15416. Ele atua como apoio à decisão: (i) classifica defeitos físicos de impressão e (ii) estima indicadores *inspirados* nos parâmetros da norma (contraste, uniformidade), sempre rotulados como aproximação, nunca como grau oficial.

1.3 Objetivos

Objetivo geral. Desenvolver e avaliar um sistema, exposto por uma API única e consumido por uma interface web responsiva — acessível de qualquer dispositivo com navegador, independentemente do sistema operacional —, capaz de detectar defeitos comuns de impressão em etiquetas de códigos de barras e sugerir a causa provável, empregando visão computacional e orquestração multiagente com o *Agent Development Kit* (ADK).

Objetivos específicos.

- a) caracterizar os defeitos de impressão mais frequentes em impressoras térmicas industriais (referência: Zebra série ZT) e mapeá-los para suas causas físicas;
- b) construir/organizar um conjunto de imagens rotuladas contemplando as classes de defeito e a classe “sem defeito”;
- c) projetar uma arquitetura multiagente que decomponha a tarefa em leitura/decodificação, estimativa de indicadores de qualidade, detecção de defeitos físicos e diagnóstico de causa-raiz fundamentado na documentação do fabricante;
- d) implementar a solução e disponibilizá-la por uma API que serve uma interface web acessível de qualquer dispositivo com navegador; e
- e) avaliar o desempenho do classificador de defeitos com métricas por classe e validação cruzada, comparando-o a arquiteturas de referência, e discutir o papel da decomposição multiagente na modularidade e na interpretabilidade do laudo.

Escopo de símbolos. O trabalho contempla **1D** (Code 128, Code 39, EAN-13/EAN-8, *Interleaved 2 of 5*, Pharmacode) e **2D** (Data Matrix, QR Code). A ênfase de qualidade recai sobre a ISO/IEC 15416 (1D) e a ISO/IEC 15415 (2D). O conjunto de imagens reais atualmente disponível cobre apenas 1D; as amostras 2D e as amostras de defeito de impressão são obtidas por geração sintética controlada (Seção 4.2).

1.4 Contribuições do trabalho

As principais contribuições são:

- a) um **mapeamento sistematizado** de defeitos visuais de impressão para causas físicas e ações corretivas, consolidado a partir da documentação técnica da Zebra (guia da ZT411/ZT421, manual de manutenção e base de conhecimento);
- b) uma **arquitetura multiagente** (ADK) que separa leitura, estimativa de indicadores, detecção de defeitos e diagnóstico de causa-raiz, servida por uma API única que alimenta uma interface web multiplataforma, acessível de qualquer dispositivo com navegador;
- c) um **agente de diagnóstico fundamentado** (com recuperação sobre a documentação do fabricante) que vai além da nota e sugere causa provável e correção;
- d) uma **avaliação experimental** do classificador de defeitos, com métricas por classe, validação cruzada estratificada e comparação a arquiteturas de referência (ResNet18 e EfficientNet-B0) sob protocolo idêntico, acompanhada de teste de significância estatística; e

- e) um **conjunto de imagens híbrido**: imagens reais de códigos de barras capturados em condições variadas (52 amostras 1D organizadas por simbologia) combinadas a um conjunto de defeitos de impressão gerados sinteticamente de forma controlada e reproduzível.

1.5 Organização do artigo

A Seção 2 apresenta a fundamentação teórica. A Seção 3 revisa trabalhos relacionados e delimita a lacuna. A Seção 4 descreve a metodologia. A Seção 5 detalha a solução proposta. A Seção 6 apresenta e discute os resultados. A Seção 7 conclui e aponta trabalhos futuros.

2 Fundamentação teórica

2.1 Códigos de barras e simbologias

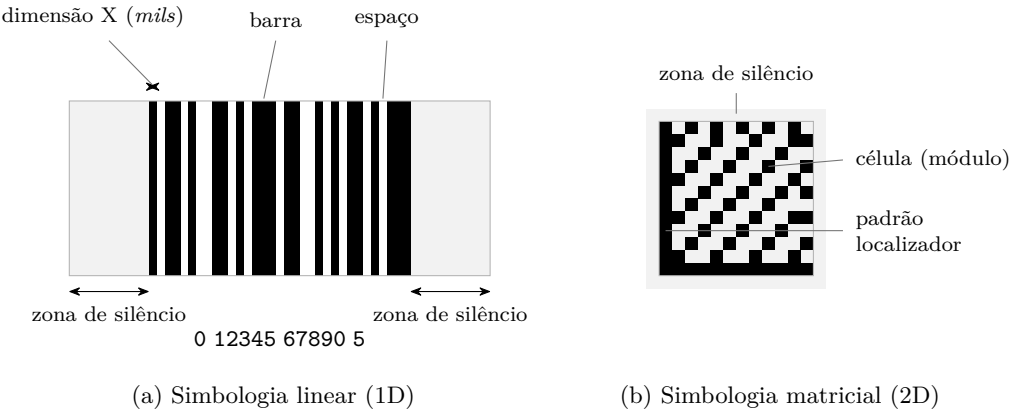
Um código de barras é uma representação óptica de dados, legível por máquina, na qual a informação é codificada no arranjo de elementos claros e escuros. O conjunto de regras que define *quais* dados podem ser codificados e *como* constitui uma **simbologia**; existem centenas delas, desenvolvidas e refinadas ao longo de décadas, muitas adaptadas a necessidades específicas de negócio ou de conformidade setorial (Seagull Scientific, 2024). As simbologias organizam-se em famílias: **lineares (1D)** — como Code 128, EAN-13 e GS1-128 —, em que a informação está na sequência horizontal de barras e espaços de larguras variáveis; **bidimensionais**, subdivididas em **empilhadas** (*2D stacked*, como o PDF417) e **matriciais** (*2D matrix*, como Data Matrix e QR Code), que distribuem os dados em uma matriz de células, permitindo maior densidade e correção de erros; e **compostas** (*composite*), que unem um componente linear a um 2D.

Alguns elementos estruturais são comuns às simbologias e determinam sua legibilidade (Figura 1). A menor largura nominal de um elemento é a **dimensão X** (ou módulo) e serve de referência para todas as tolerâncias; seu menor valor prático — em geral expresso em *mils* (milésimos de polegada) — é limitado pela resolução da impressora (Seagull Scientific, 2024). O símbolo é delimitado por **zonas de silêncio** (áreas claras) obrigatórias antes e depois das barras, sem as quais o leitor não isola o código. Somam-se a eles os **caracteres de início/fim**, que sinalizam ao leitor os limites e a orientação; o **dígito verificador**, mecanismo opcional (dependente da simbologia) de detecção de erro; e a **linha de interpretação legível por humano** (HRI), que reproduz em texto o conteúdo codificado (Seagull Scientific, 2024).

A variedade de simbologias reflete diferentes compromissos entre tipo e volume de dados, tamanho, densidade e setor de aplicação. O Quadro 1 sintetiza as mais difundidas.

Duas camadas de padronização coexistem. As **normas técnicas** especificam a simbologia em si — sua codificação e geometria (por exemplo, a ISO/IEC 15417 para o Code 128 ou a ISO/IEC 16022 para o Data Matrix) —, enquanto as **normas de aplicação**, independentes da simbologia, definem que dados e estrutura cada setor exige (Seagull Scientific, 2024). Sobre essa base atua o sistema **GS1**, cuja chave é o **GTIN** (*Global Trade Item Number*), descrito pela própria GS1 como “a chave para gerenciar produtos em toda a cadeia de valor”, da fabricação ao consumidor final (GS1 Brasil, 2024). Essa padronização resulta de décadas de trabalho de organismos como UCC, ANSI, CEN, ISO/IEC e a AIM: a própria avaliação de qualidade de impressão evoluiu do método ANSI (X3.182-1990) para a norma europeia EN 1635 (1995) e,

Figura 1 – Anatomia de símbolos de código de barras: elementos de uma simbologia linear (1D) e de uma matricial (2D)



Fonte: elaboração própria (2026).

Quadro 1 – Principais simbologias de código de barras

Simbologia	Categoria	Tipo de dado	Aplicação típica	Norma técnica
UPC-A / EAN-13	Linear (1D)	Numérico	Varejo / ponto de venda (GS1)	GS1 Gen. Specs.
Code 39	Linear (1D)	Alfanumérico	Industrial, defesa (MIL-STD-130)	ISO/IEC 16388
Code 128	Linear (1D)	Alfanumérico (ASCII)	Logística e uso geral	ISO/IEC 15417
GS1-128	Linear (1D)	Alfanumérico	Unidades logísticas / envio	ISO/IEC 15417 + GS1
ITF-14	Linear (1D)	Numérico	Caixas e embalagens (DUN-14)	ISO/IEC 16390
GS1 DataBar	Linear (1D)	Numérico (GTIN)	Itens pequenos, perecíveis	GS1
PDF417	2D empilhada	Alfanum. binário	/ Documentos, identidade, transporte	ISO/IEC 15438
Data Matrix	2D matricial	Alfanum. binário	/ Marcação de peças, saúde	ISO/IEC 16022
QR Code	2D matricial	Alfanum. binário	/ Consumo, <i>mobile</i> , rastreo	ISO/IEC 18004
Aztec	2D matricial	Alfanum. binário	/ Bilhetagem, transporte	ISO/IEC 24778

Fonte: elaboração própria (2026), a partir de [Seagull Scientific](#) (2024).

então, para a internacional ISO/IEC 15416 (2000), todas partilhando a mesma metodologia ([AIM Inc.](#), 2012) — assunto aprofundado na Seção 2.2.

Esse arcabouço sustenta um alcance operacional vasto. A **precisão de dados** é a motivação mais comum para adotar um sistema de código de barras: empresas com leitura automatizada alcançam comumente **99% de acurácia**, ante cerca de 85% na entrada manual, aproximando-se da meta de 100% exigida em contextos críticos como hospitais e manufatura ([Zebra Technologies](#),

2013). A leitura óptica funciona porque “as barras pretas absorvem a luz e os espaços brancos a refletem” (GS1 Brasil, 2024), convertendo em segundos uma ação física em transação digital. Ganhos como acurácia, velocidade, rastreabilidade e controle de inventário explicam a presença dos códigos de barras em praticamente todos os setores (Quadro 2).

Quadro 2 – Setores e usos representativos da etiquetagem por código de barras

Setor	Uso representativo
Varejo	Ponto de venda, precisão de inventário e <i>fulfillment omnichannel</i>
Transporte e logística	Comprovante de entrega, inventário de armazém e visibilidade da cadeia
Manufatura	Rastreio de produção (<i>work-in-progress</i>) e rastreabilidade de peças
Saúde	Identificação de pacientes, amostras e medicamentos
Governo	Gestão de ativos, licenciamento e resposta a emergências
Energia e utilidades	Rastreio de ativos e ordens de serviço em campo

Fonte: elaboração própria (2026), a partir de Zebra Technologies (2024a) e GS1 Brasil (2024).

Em todos esses cenários, contudo, o benefício depende de uma premissa simples: o símbolo impresso precisa ser lido de forma confiável logo na primeira tentativa. A leitura pressupõe que as proporções impressas correspondam às nominais; pequenos desvios sistemáticos — barras mais largas (**ganho**) ou mais estreitas (**perda**) que o previsto — comprometem a decodificação. É exatamente esse tipo de desvio que a impressão térmica pode introduzir e cujo diagnóstico é o objeto deste trabalho.

2.2 Qualidade de impressão e verificação (ISO/IEC 15416 e 15415)

A avaliação normativa parte do **perfil de refletância de varredura** (*scan reflectance profile*, SRP): uma linha de varredura atravessa o símbolo medindo a refletância ao longo do percurso, produzindo uma curva de picos (espaços claros) e vales (barras escuras). Todos os parâmetros de nota derivam desse perfil (ISO/IEC, 2016).

Para símbolos lineares, a ISO/IEC 15416 avalia nove atributos por varredura, entre os quais se destacam os apresentados na Tabela 1.

O ajuste de *darkness* (temperatura da cabeça) é um dos fatores que mais afetam esses parâmetros. A Figura 2 ilustra a transição da impressão clara demais (*too light*), em que as barras finas desaparecem, à escura demais (*too dark*), em que barras e espaços se fundem, passando pela faixa dentro da especificação (*in spec*).

A nota de uma varredura é a **menor** entre os parâmetros; a nota do símbolo é a **média de dez varreduras** distribuídas ao longo da altura. Para 2D, a ISO/IEC 15415 cumpre papel equivalente com parâmetros adaptados à matriz (ISO/IEC, 2024), e a marcação direta de peça (DPM) é tratada pela ISO/IEC 29158 (ISO/IEC, 2020).

Ponto metodológico importante. A verificação *conformante* exige condições ópticas controladas — fonte de luz definida, abertura de medição, geometria e calibração de refletância — providas por verificadores certificados (ISO/IEC, 2006). Uma imagem de câmera comum não

Tabela 1 – Parâmetros de qualidade da ISO/IEC 15416 e sua relação com o defeito de impressão

Parâmetro	O que mede	Relação com o defeito de impressão
Contraste do símbolo (SC)	Diferença entre a maior e a menor refletância	Impressão clara ou mídia- <i>ribbon</i> incompatível reduzem o contraste
Contraste de borda (EC)	Contraste entre barra e espaço adjacentes	Bordas “borradas” por pressão/velocidade inadequadas
Modulação (MOD)	Uniformidade dos elementos ao longo do símbolo	Pressão desigual e sujeira geram variação
Defeitos (ERN)	Maior não uniformidade de refletância de um elemento	Manchas, <i>voids</i> e pontos queimados
Decodificabilidade	Margem do símbolo frente ao algoritmo de referência	Ganho/perda de largura das barras

Fonte: elaboração própria (2026), a partir de [ISO/IEC \(2016\)](#).

reproduz essas condições. Por isso, neste trabalho os parâmetros da norma são usados como referencial conceitual e como indicadores aproximados, não como grau oficial — o que delimita honestamente o escopo (Seção 2).

2.3 Defeitos típicos de impressão térmica e suas causas

Impressoras térmicas de transferência (como a Zebra ZT411/ZT421) formam a imagem aquecendo seletivamente uma cabeça de impressão que transfere tinta do *ribbon* para a mídia, prensada pelo rolete (*platen*). Cada componente falha de um modo com assinatura visual característica. A partir do guia do usuário da ZT411/ZT421, do procedimento de manutenção e da base de conhecimento da Zebra ([Zebra Technologies, 2024d](#)), consolida-se o mapeamento da Tabela 2 — que é, em si, uma das contribuições do trabalho.

A Figura 3 exemplifica, em etiquetas impressas reais, as assinaturas visuais de vários desses defeitos, tal como documentadas pela Zebra ([Zebra Technologies, 2024b](#)). Note-se a correspondência com as classes reproduzidas sinteticamente na Seção 4.2.

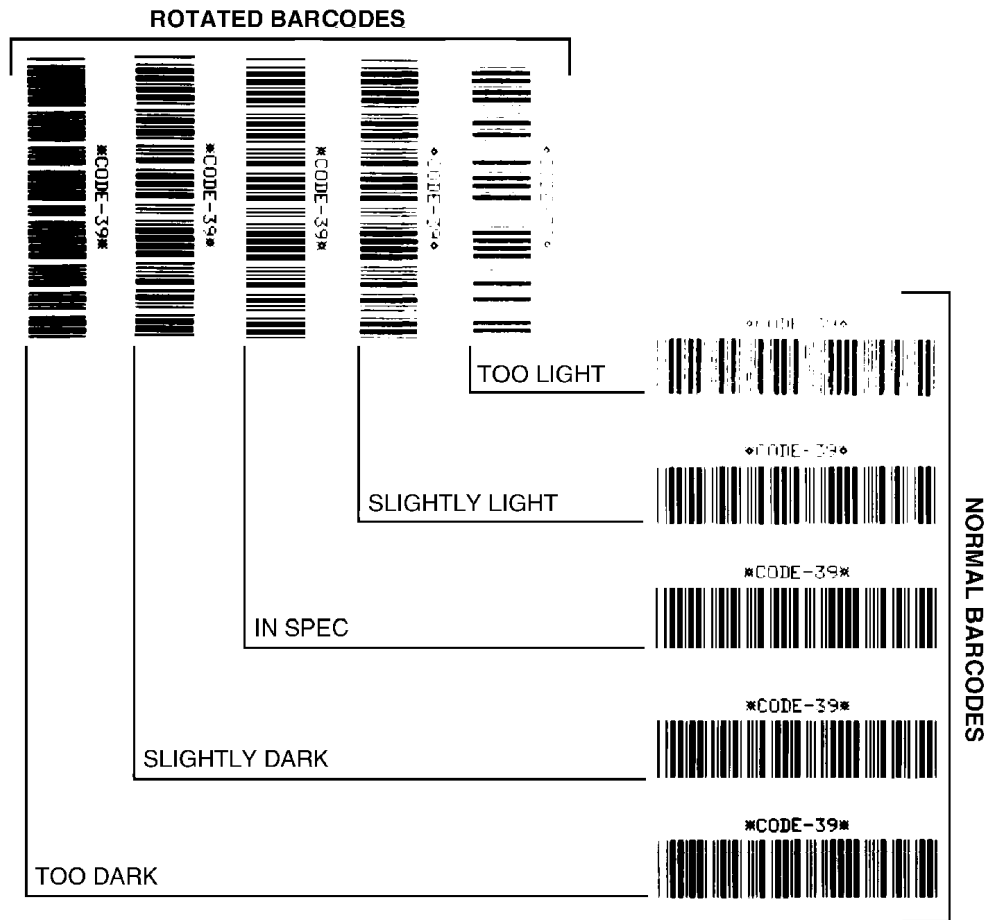
A manutenção preventiva atua sobre as causas de origem: a limpeza periódica da cabeça e do rolete com álcool isopropílico 99,7% (ou kit de manutenção Zebra) é indicada quando surgem *voids*, e a documentação alerta que *darkness* excessivo desgasta a cabeça prematuramente e pode “queimar” o *ribbon* ([Zebra Technologies, 2024d](#)). Esse conhecimento é o que alimenta o agente de diagnóstico proposto na Seção 5.

2.4 Processamento de imagens e visão computacional para inspeção

O reconhecimento dos defeitos acima é um problema de **inspeção visual automatizada**. As etapas clássicas envolvem:

- pré-processamento** — conversão de cor, correção de iluminação e realce de contraste para atenuar variações de captura ([GONZALEZ; WOODS, 2018](#));
- localização/segmentação do símbolo** — isolar a região do código na etiqueta, condição para medir refletância e detectar defeitos localizados ([SZELISKI, 2022](#));

Figura 2 – Efeito do *darkness* na qualidade do código de barras: do claro demais (*too light*) ao escuro demais (*too dark*)



Fonte: [Zebra Technologies \(2024c\)](#).

- c) **detecção de objetos/defeitos** — localizar e classificar regiões defeituosas; abordagens supervisionadas de um estágio (por exemplo, da família YOLO) são amplamente usadas em inspeção industrial;
- d) **classificação por redes convolucionais (CNN)** — atribuir a classe de defeito a partir de características aprendidas, dispensando extração manual de atributos; e
- e) **detecção de anomalias** — quando amostras de defeito real são escassas, métodos não supervisionados aprendem o “normal” e sinalizam desvios, contornando o desbalanceamento de dados.

A literatura recente de inspeção industrial converge nesses pilares e destaca um desafio recorrente diretamente relevante a este trabalho: a **escassez de imagens de defeito real**, que motiva aumento de dados e abordagens semi/não supervisionadas ([SHUKLA et al., 2025](#); [TAO et al., 2022](#)).

2.5 Redes convolucionais, MobileNetV3 e transferência de aprendizado

Uma **rede neural convolucional (CNN)** aprende, camada a camada, filtros que são convolvidos com a imagem para produzir *mapas de características (feature maps)*. As primeiras

Tabela 2 – Mapeamento entre aparência do defeito, causa provável e ação corretiva

Defeito (aparência)	Causa provável	Ação corretiva típica
Código não escaneia; imagem “suja”	<i>Darkness</i> alto demais ou pressão da cabeça incorreta	Reduzir o <i>darkness</i> ; reduzir velocidade; ajustar pressão
Impressão clara (<i>faded</i>) em toda a etiqueta	<i>Darkness</i> baixo, mídia/ <i>ribbon</i> incompatível ou alta velocidade	Elevar <i>darkness</i> ; trocar para combinação mídia- <i>ribbon</i> adequada
Impressão clara/escura em um lado da etiqueta	Pressão desigual da cabeça	Ajustar a pressão da cabeça (<i>toggles</i>)
Linhas cinza finas e angulares em etiquetas em branco	<i>Ribbon</i> enrugado	Corrigir tensão/alinhamento do <i>ribbon</i>
Trilhas longas de impressão faltando	Elemento da cabeça danificado (linhas brancas verticais)	Acionar assistência técnica (troca de cabeça)
<i>Voids</i> no código; qualidade inconsistente	Cabeça de impressão suja	Limpar cabeça e rolete com álcool isopropílico 99,7%
Manchas (<i>smudge</i>)	Mídia/ <i>ribbon</i> inadequados para alta velocidade	Trocar por insumos recomendados
Perda de registro / deslocamento vertical	Roleta sujo, mídia mal carregada, sensor descalibrado	Limpar rolete; recarregar mídia; calibrar sensores

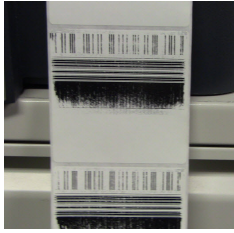
Fonte: elaboração própria (2026), a partir de [Zebra Technologies \(2024d\)](#).

camadas respondem a padrões genéricos — bordas, cantos e texturas —, enquanto as camadas profundas combinam esses padrões em conceitos específicos da tarefa; é essa hierarquia que dispensa a extração manual de atributos ([GONZALEZ; WOODS, 2018](#); [SZELISKI, 2022](#)). Como as camadas iniciais aprendem características transferíveis entre domínios, é possível reaproveitar uma rede pré-treinada em um grande conjunto (ImageNet) e apenas **ajustá-la** (*fine-tuning*) à tarefa-alvo — estratégia de **transferência de aprendizado** que melhora a generalização quando há poucos dados rotulados ([YOSINSKI et al., 2014](#)).

Adota-se a **MobileNetV3-Small** ([HOWARD et al., 2019](#)), projetada para ser leve e eficiente. Três elementos explicam sua economia: (i) **convoluções separáveis em profundidade** (*depthwise separable*), que fatoram a convolução padrão em um filtro espacial por canal seguido de uma combinação 1×1 , reduzindo drasticamente parâmetros e operações; (ii) blocos **Squeeze-and-Excitation**, um mecanismo de atenção que reponderar canais conforme sua relevância; e (iii) a não linearidade **h-swish**, aproximação barata do *swish*. O resultado é uma rede de $\approx 1,5$ milhão de parâmetros que treina e infere bem mesmo em CPU — adequada ao objetivo de baixo custo deste trabalho. Como *baselines* de comparação (Seção 6) empregam-se a **ResNet18** ([HE et al., 2016](#)), que populariza as conexões residuais, e a **EfficientNet-B0** ([TAN; LE, 2019](#)), que escala profundidade, largura e resolução de forma balanceada.

Generalização com poucos dados. Treinar em um conjunto pequeno favorece o *overfitting* (o modelo memoriza o treino e não generaliza). Três mecanismos o mitigam aqui: a transferência de aprendizado (reduz os parâmetros efetivamente estimados do zero), o **aumento de dados** (*data augmentation*) — rotações e variações de brilho/contraste que ampliam a variabilidade

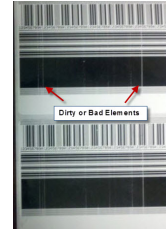
Figura 3 – Assinaturas visuais reais de defeitos de impressão térmica em etiquetas de códigos de barras



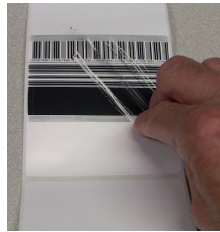
(a) Impressão clara (*faded*)



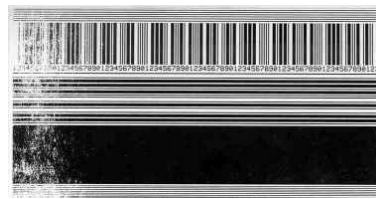
(b) *Ribbon* enrugado



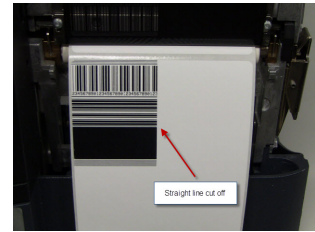
(c) Elemento da cabeça danificado



(d) Mancha (*smear*)



(e) Pressão desigual da cabeça



(f) Perda de registro (*cutoff*)

Fonte: [Zebra Technologies \(2024b\)](#).

sem alterar o rótulo ([SHORTEN; KHOSHGOFTAAR, 2019](#)) — e a própria arquitetura leve, de menor capacidade. A avaliação por **validação cruzada** (Seção 6) complementa essas medidas ao estimar o desempenho de forma menos sensível a uma única partição.

2.6 Sistemas multiagentes e o *Agent Development Kit* (ADK)

Um **sistema multiagente** é um conjunto de agentes autônomos que colaboram para uma meta comum, cada um responsável por uma sub tarefa. O *Agent Development Kit* (ADK), apresentado pelo Google no Cloud NEXT 2025, é um *framework* de código aberto para construir e orquestrar esses sistemas ([Google, 2025b](#)). Ele organiza os agentes em três tipos:

- a) **agentes LLM** — o “raciocínio”, que interpreta entradas e decide ações;
- b) **agentes de *workflow*** — orquestradores de fluxo (sequencial, paralelo, em laço); e
- c) **agentes customizados** — especialistas que encapsulam lógica ou modelos próprios (por exemplo, um modelo de visão).

Os agentes se organizam em uma **hierarquia** e se comunicam por estado compartilhado, delegação e invocação explícita ([Google, 2025a](#)). A justificativa central do paradigma — que motiva a arquitetura adotada neste trabalho — é que decompor um problema em agentes especializados tende a ser mais confiável, modular e manutenível do que um único modelo/prompt monolítico, porque cada agente resolve uma tarefa mais simples e bem delimitada.

3 Trabalhos relacionados

3.1 Verificação normativa e diagnóstico de defeitos

A base da verificação de qualidade são as normas ISO/IEC 15416 e 15415, implementadas por verificadores comerciais que reportam notas A–F ([ISO/IEC, 2016](#); [ISO/IEC, 2024](#)). Há também

soluções que inferem falhas do mecanismo de impressão a partir da própria leitura — por exemplo, detectando linhas não impressas para gerar um relatório de manutenção do cabeçote e alertar antes da falha total (ACKLEY, 2017). **Limitação comum:** dependem de *hardware* dedicado e/ou entregam a nota/alerta, sem um diagnóstico de causa-raiz acionável e contextualizado pela documentação do fabricante.

3.2 Inspeção visual de defeitos com aprendizado profundo

A inspeção industrial baseada em aprendizado profundo é sistematizada em levantamentos recentes que cobrem CNNs, detecção de objetos e detecção de anomalias, com ênfase no problema de dados escassos (SHUKLA et al., 2025; TAO et al., 2022). Aplicações diretas ao domínio de impressão incluem a detecção de defeitos em tecidos estampados com CNN (JUN et al., 2021) e a detecção/localização de anomalias em manufatura aditiva por transferência de aprendizado (AVRO et al., 2024). **Limitação comum:** focam a classificação/localização do defeito visual, sem ligá-lo à causa física do processo nem a uma ação corretiva.

3.3 Sistemas multiagentes aplicados a tarefas de visão

A orquestração multiagente com ADK e *frameworks* correlatos vem sendo aplicada a tarefas complexas que se beneficiam de decomposição, tratando agentes especializados como ferramentas de um agente raiz (Google, 2025b). **Lacuna observada:** a combinação de orquestração multiagente com inspeção de qualidade de impressão de códigos de barras — em especial acoplando detecção visual a diagnóstico fundamentado em documentação técnica — é pouco explorada.

3.4 Lacuna que este trabalho preenche

Reunindo os três eixos: os verificadores dão nota mas não causa; os métodos de visão detectam o defeito mas não a causa nem a correção; e a orquestração multiagente é madura porém raramente aplicada a este domínio. Este trabalho se posiciona nessa interseção, propondo um sistema de baixo custo, servido por API, que combina estimativa de indicadores de qualidade, detecção visual de defeitos físicos e diagnóstico de causa-raiz fundamentado na documentação da Zebra, orquestrados por agentes especializados. O Quadro comparativo da Tabela 3 posiciona este trabalho frente às abordagens existentes.

Tabela 3 – Comparativo entre abordagens quanto aos recursos oferecidos

Abordagem	Nota ISO	Defeito físico	Causa-raiz	Sem <i>hardware</i> dedicado
Verificadores certificados	Sim	Não	Não	Não
Visão computacional (CNN/anomalia)	Não	Sim	Não	Sim
Detecção via leitura (parte)	Parcial	Parcial	Parcial	Não
Este trabalho	Aprox.	Sim	Sim	Sim

Fonte: elaboração própria (2026).

3.5 Comparação explícita com *Research on a Multi-Type Barcode Defect Detection Model Based on Machine Vision*

Recentemente, Duan et al. (DUAN et al., 2025) propuseram um sistema de detecção de defeitos em códigos de barras baseado em uma arquitetura em dois estágios composta por YOLOv8n e uma rede híbrida ResNet50–ViT (denominada **Y8-LiBARNet**), treinada sobre o conjunto público **BarBeR**. Embora o método apresente elevada precisão na localização e classificação de códigos defeituosos, sua saída limita-se à identificação do estado do código (íntegro ou defeituoso), sem estabelecer relação entre o defeito observado, sua provável causa física e a ação corretiva correspondente. O presente trabalho complementa essa lacuna ao integrar classificação visual, diagnóstico fundamentado na documentação técnica do fabricante e orquestração multiagente para apoio à manutenção industrial. Os dois trabalhos atacam problemas parecidos, mas por **perspectivas diferentes**, o que reduz a sobreposição científica; a Tabela 4 confronta ambos por aspecto.

Tabela 4 – Comparação por aspecto entre este trabalho e Duan et al. (2025)

Aspecto	Este trabalho	Duan et al. (DUAN et al., 2025)
Objetivo	Diagnosticar defeitos de impressão e sugerir a causa física	Detectar códigos e classificar se possuem defeito
Entrada	Foto de uma etiqueta	Foto contendo um ou mais códigos
Pipeline	OpenCV + CNN + multiagentes + RAG	YOLOv8 + ResNet50 + ViT
Saída	Defeito + causa + ação corretiva	Localização + classe (íntegro/defeituoso)
Foco Hardware alvo	Apoio à manutenção da impressora CPU comum (sem GPU)	Inspeção industrial automática GPU dedicada (RTX 4060 nos experimentos)
Dataset	Defeitos sintéticos sobre códigos-base públicos (BarBeR/Roboflow) + imagens reais próprias	BarBeR (8.748 imagens públicas, com rótulos reais)

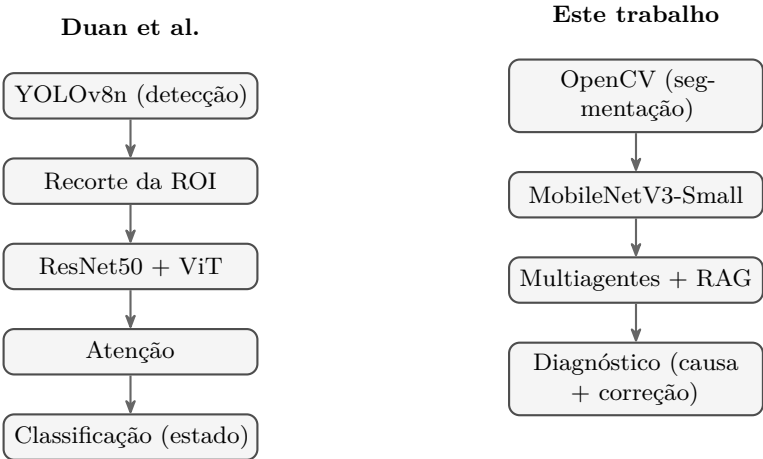
Fonte: elaboração própria (2026).

Onde o método de Duan et al. é mais forte. Há pontos em que aquele trabalho é objetivamente superior. **(i) Dataset:** utilizam cerca de 8.748 imagens do BarBeR, contemplando 18 simbologias, códigos 1D e 2D, iluminação variada, superfícies planas e curvas e diferentes sensores, **com rótulos reais de defeito**; este trabalho apoia-se em 52 imagens reais somadas ao conjunto sintético de defeitos — este último gerado sobre códigos-base limpos das mesmas bases públicas (BarBeR (VEZZALI et al., 2024) e Roboflow (Roboflow Universe, 2024)), porém **sem herdar seus rótulos** —, totalizando cerca de 630 imagens para treino/teste da CNN — provável primeiro ponto a ser observado por um revisor. **(ii) Arquitetura:** adotam um *pipeline* de dois estágios mais sofisticado (Figura 4), ao passo que a proposta aqui é deliberadamente mais simples, priorizando baixo custo. **(iii) Resultados:** reportam valores superiores em termos absolutos (Tabela 5). À primeira vista o desempenho parece inferior, mas a comparação direta seria injusta, pois as **tarefas são distintas** — decidir a *presença* do defeito em imagens reais *versus* classificar a *causa* física sobre um conjunto sintético.

Onde este trabalho é mais forte. A maior oportunidade está no escopo. **(i) Problema distinto:** enquanto o trabalho de referência responde “*existe* defeito?”, este responde “*qual* defeito é este?” e, além disso, “*qual componente* da impressora provavelmente o causou?” — passo

que simplesmente não existe no trabalho comparado. **(ii) Diagnóstico de causa-raiz:** este é o principal diferencial. A solução mapeia imagem → classe do defeito → documentação Zebra → causa provável → ação corretiva (Figura 5), aproximando-se de um sistema especialista apoiado por IA; o trabalho de Duan et al. encerra-se na classificação. **(iii) Orquestração multiagente:** não há, naquele trabalho, equivalente ao *pipeline* de agentes especializados (leitura → indicadores → CNN → RAG → laudo) da Figura 7, que converte um classificador em um sistema de apoio à decisão. **(iv) Interpretabilidade e custo:** o sistema explica por que detectou o defeito, qual documento sustentou a conclusão, qual componente trocar e qual ajuste fazer — informação de valor industrial direto — e o faz em **CPU comum, sem GPU**, ao passo que o trabalho de referência recorre a uma GPU dedicada (RTX 4060); a viabilidade em hardware modesto é, por si só, um diferencial para o cenário de baixo custo visado aqui.

Figura 4 – Contraste entre os *pipelines*: Duan et al. (dois estágios) e este trabalho



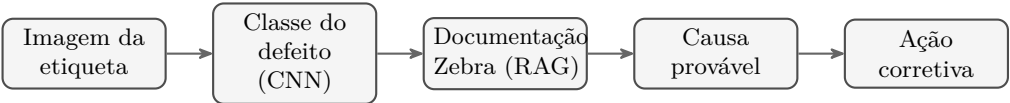
Fonte: elaboração própria (2026).

Tabela 5 – Comparação quantitativa (valores absolutos; tarefas e dados *distintos*)

Trabalho	Tarefa	Dataset	Métrica	Valor
Duan et al. (DUAN et al., 2025)	Localização	BarBeR (real)	mAP@0,5	0,984
Duan et al. (DUAN et al., 2025)	Classificação	BarBeR (real)	Acurácia / F1	0,925 / 0,919
Este trabalho	Classificação (7 causas)	Sintético	Acurácia / Macro-F1	0,84 / 0,84

Fonte: elaboração própria (2026). A comparação direta seria injusta: as tarefas (*presença vs. causa* do defeito), os conjuntos (real *vs.* sintético) e o *hardware* (GPU *vs.* CPU) diferem; os valores apenas situam a ordem de grandeza.

Figura 5 – Cadeia de diagnóstico de causa-raiz que estende a classificação — ausente em Duan et al.



Fonte: elaboração própria (2026).

Comparando apenas o classificador. A escolha do *backbone* apoiou-se em uma etapa

de **seleção de modelos** executada durante a construção deste trabalho (revisão V2, 15 jul. 2026), sobre o conjunto sintético (Seção 4.2). Três arquiteturas foram treinadas e avaliadas sob *protocolo idêntico* — mesmos dados, augmentação e hiperparâmetros —, com *tuning* e treinamento completos; a Tabela 6 resume os resultados atingidos. Esse é um rigor experimental frequentemente ausente em trabalhos aplicados, e a análise completa consta da Seção 6.3.

Tabela 6 – Comparação de arquiteturas do classificador no teste fixo (protocolo idêntico)

Modelo	Acurácia	Macro-F1	Parâmetros (M)	Latência (ms)
MobileNetV3-Small (adotado)	0,84	0,84	1,5	4,5
ResNet18	0,79	0,80	11,2	17,8
EfficientNet-B0	0,90	0,89	4,0	16,5

Fonte: elaboração própria (2026). Etapa de seleção de modelos (revisão V2, 15 jul. 2026). Latência média por imagem em CPU.

- a) **significância estatística (McNemar):** nenhuma diferença é significativa ao nível de 5% — MobileNetV3-Small *vs.* EfficientNet-B0 ($b=4$, $c=10$, $p=0,18$) e *vs.* ResNet18 ($b=10$, $c=5$, $p=0,30$). Ou seja, no conjunto de teste disponível não há evidência de que a EfficientNet-B0 seja superior; somado à sua eficiência muito maior, isso sustenta a escolha da MobileNetV3-Small para o cenário de baixo custo;
- b) **validação cruzada (5 dobras estratificadas):** a MobileNetV3-Small obteve acurácia de $0,82 \pm 0,01$ e Macro-F1 de $0,82 \pm 0,02$, indicando desempenho estável e pouco sensível à partição; e
- c) **ponto fraco:** confusão mútua entre *sem defeito* e *cabeça queimada* ($F1 \approx 0,56$) — as zonas claras entre as barras imitam as linhas brancas verticais do dano na cabeça.

Novos testes, após *tuning* e treinamento, confirmaram esses resultados, consolidando a MobileNetV3-Small como o modelo adotado no trabalho.

Síntese. Em síntese, os dois trabalhos são **complementares**: o de Duan et al. é mais forte em detecção visual e validação em larga escala, enquanto este se destaca por transformar a classificação em um sistema de apoio à manutenção, combinando visão computacional, conhecimento técnico do fabricante e orquestração multiagente. Esse posicionamento constitui um diferencial claro, retomado na discussão dos resultados (Seção 6). Uma comparação *indireta* e mais justa passa por avaliar o classificador sobre o próprio BarBeR (experimento E4, Seção 6.4).

4 Metodologia

4.1 Tipo de pesquisa

Pesquisa de natureza **aplicada**, abordagem **quantitativa** e objetivo **exploratório-experimental**: constrói-se um artefato (o sistema) e mede-se seu desempenho em condições controladas, com procedimento experimental.

4.2 Conjunto de dados

O problema tem duas naturezas distintas — a **legibilidade da captura** (a imagem do código está boa o suficiente para ler?) e o **defeito de impressão** (que falha do processo produziu a marca?) — e cada uma exige um tipo de dado. Por isso, adota-se uma estratégia de **duas trilhas**.

Trilha A — imagens reais de captura (base disponível). Conjunto de 52 imagens reais de códigos de barras fotografados em produtos e embalagens, em tons de cinza, sob ângulo, reflexo, curvatura e fundos variados, já organizadas por simbologia, conforme a Tabela 7. Esse conjunto é usado para treinar/avaliar o agente de leitura/decodificação e o agente de indicadores de legibilidade. Como são todas 1D, as amostras 2D são complementadas na Trilha B.

Tabela 7 – Trilha A — imagens reais por simbologia

Simbologia	N.º de imagens
EAN-13	19
Code 39	6
Code 128	5
EAN-8	5
<i>Interleaved 2 of 5</i>	5
EAN-13 (<i>add-on 2</i>)	4
EAN-13 (<i>add-on 5</i>)	4
Pharmacode	4
Total	52

Fonte: elaboração própria (2026).

Trilha B — defeitos de impressão sintéticos (gerados). Para treinar o agente de defeitos físicos não há um conjunto público rotulado de defeitos de impressão térmica, e as fotos da base de conhecimento da Zebra são material protegido por direitos autorais. Adota-se, então, **geração sintética controlada**: a partir de **códigos de barras limpos (1D e 2D)** — extraídos de bases públicas de imagens reais, o repositório de *benchmarking* **BarBeR** (VEZZALI et al., 2024; VEZZALI et al., 2025) e projetos do **Roboflow Universe** (Roboflow Universe, 2024) — aplicam-se transformações que reproduzem a assinatura visual de cada defeito descrito na documentação Zebra (Zebra Technologies, 2024d), conforme a Tabela 8. As imagens dessas bases entram **exclusivamente como códigos-base limpos** (substrato) sobre os quais o gerador injeta os defeitos; **nenhum rótulo de defeito é herdado** dessas fontes. Um gerador de referência foi implementado (`generate_dataset.py`) e produz as sete classes de forma balanceada, com divisão automática em treino/validação/teste (70/15/15) e parâmetros de defeito amostrados aleatoriamente em faixas definidas. A Figura 6 ilustra amostras das classes geradas.

Essa abordagem é reproduzível, balanceável e livre de restrições autorais, e é prática consolidada na literatura de inspeção quando amostras reais são escassas (SHUKLA et al., 2025; TAO et al., 2022).

Divisão e aumento de dados. Cada trilha é dividida em treino/validação/teste (70/15/15), estratificando por classe. Sobre a Trilha A aplica-se aumento de dados clássico (rotação, variação de brilho/contraste, recortes) para ampliar a variabilidade de captura. Os parâmetros dos defeitos

Tabela 8 – Trilha B — classes de defeito e transformação sintética correspondente

Classe de defeito	Transformação sintética	Sintoma reproduzido
Cabeça queimada (elemento danificado)	Linhas brancas verticais contínuas	Trilhas longas de impressão faltando
<i>Ribbon</i> enrugado	Faixas claras finas e diagonais	Linhas cinza angulares
Ponto queimado / <i>darkness</i> alto	Manchas escuras localizadas	Excesso de temperatura
Impressão clara (<i>darkness</i> baixo)	Redução de contraste global	<i>Faded</i> / <i>mídia-ribbon</i> incompatível
Pressão desigual	Borrão direcional (gradiente em um lado)	Densidade não uniforme
Cabeça suja (<i>voids</i>)	Falhas brancas pontuais nas barras	<i>Void</i> s no código
Sem defeito	(código limpo)	Classe de controle

Fonte: elaboração própria (2026).

sintéticos (intensidade, quantidade, posição) são amostrados aleatoriamente em faixas definidas, para evitar que o modelo aprenda um padrão fixo.

Datasets externos e proveniência das imagens-base. Os códigos-base limpos da Trilha B provêm de duas fontes públicas de imagens reais de códigos de barras: o repositório de *benchmarking* **BarBeR** (VEZZALI et al., 2024; VEZZALI et al., 2025) e projetos do **Roboflow Universe** (Roboflow Universe, 2024). Em ambos os casos as imagens entram **apenas como substrato limpo** para a síntese de defeitos (papel **bases**); opcionalmente, uma partição real é reservada como **conjunto de teste de generalização** (sintético → real, papel **test**), nunca misturada ao treino. O projeto inclui um utilitário de *download* reproduzível — por exemplo, para baixar e recortar os códigos-base de um projeto do Roboflow:

```
python -m config.download_roboflow -workspace o-xfs34
      -project barcode-detection-tvdug -version 1
      -role bases -symbolology mixed -crop
```

Licenciamento das imagens. As imagens são utilizadas **em conformidade com as licenças declaradas nas respectivas fontes**: a licença do repositório BarBeR (VEZZALI et al., 2024; VEZZALI et al., 2025) e a licença de cada projeto do Roboflow Universe — majoritariamente *Creative Commons* CC BY 4.0, verificada projeto a projeto na página de origem (Roboflow Universe, 2024). O uso restringe-se a **códigos-base para geração sintética de defeitos**, com a devida atribuição aos autores e sem redistribuição das imagens originais.

4.3 Ferramentas utilizadas

- a) **visão computacional:** Python, OpenCV e **PyTorch**. O PyTorch é hoje o *framework* predominante na comunidade acadêmica, o que favorece a reprodutibilidade e a comparação com trabalhos correlatos. Para atender a máquinas modestas, adota-se transferência de aprendizado com o *backbone* leve **MobileNetV3-Small** (ResNet18 e EfficientNet-B0 são usados como referências de comparação na Seção 6), que treina bem em GPU de entrada

ou mesmo CPU; como a inferência é servida pela API (lado servidor), o dispositivo do usuário não precisa executar o modelo;

- b) **decodificação:** `pyzbar` (ZBar) para decodificar o símbolo (simbologia e conteúdo) e verificar a legibilidade, com os detectores nativos do OpenCV (`barcode.BarcodeDetector` e `QRCodeDetector`) como *fallback* de detecção/leitura;
- c) **orquestração multiagente:** *Agent Development Kit* (ADK) (Google, 2025b), com **Gemini** como modelo LLM dos agentes de raciocínio, aproveitando a integração nativa do ADK com o ecossistema Google;
- d) **entrega:** API única (FastAPI) que serve um **chat** web internacionalizado (português/inglês) e responsivo para o envio da imagem, acessível de qualquer dispositivo com navegador — computador, *notebook*, *tablet* ou celular, em qualquer sistema operacional; e
- e) **redação do documento de pesquisa:** parte do trabalho consistiu em garantir que o documento fosse compatível com $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ e com as normas da ABNT (via classe `abnTeX2`) e que pudesse ser importado no **Overleaf** — ferramenta amplamente utilizada pela comunidade acadêmica e científica.

Decisão de entrega: solução universal multiplataforma. A solução foi concebida, desde o princípio, como uma ferramenta **universal e multiplataforma**, entregue exclusivamente pela web. Essa escolha decorre da própria arquitetura: como todo o processamento pesado (visão computacional, CNN e orquestração multiagente) reside no servidor, atrás de uma API única, o cliente torna-se **leve e sem estado** — cabe-lhe apenas capturar a imagem e apresentar o laudo. Concentrar essa camada em um *front-end* web responsivo e internacionalizado significa manter **um único código de interface** que atende a qualquer computador, *notebook*, *tablet* ou celular — com iOS, Android ou qualquer outro sistema operacional — diretamente pelo navegador, sem instalação nem manutenção de binários por plataforma. A decisão é orientada por **experiência de uso** (UX/UI): reduzir o atrito de adoção a um clique amplia o alcance da ferramenta para além do chão de fábrica e dos coletores industriais Android, aproximando a solução de um **produto efetivamente aplicável** aos cenários que se propõe a cobrir.

Correspondência com o conteúdo da disciplina. As técnicas empregadas exercitam, de ponta a ponta, o pipeline clássico de Visão Computacional visto na disciplina, culminando em uma CNN. A Tabela 9 mapeia cada tópico ao ponto concreto em que foi aplicado na solução — todos verificáveis no módulo `inspector.vision` e na CNN (`inspector.network`).

4.4 Ambiente experimental

Os experimentos foram executados em **CPU** (Intel Core i7-1185G7, 8 *threads*, 31 GB de RAM), **sem GPU dedicada**, com Python 3.12, PyTorch 2.13 (CPU), *torchvision* 0.28 e OpenCV 4.10; o ambiente Python é gerenciado com `uv`. O *fine-tuning* da CNN sobre o conjunto sintético levou poucos minutos nessa configuração, evidenciando a viabilidade em máquinas modestas. *Observação:* como o contraste medido depende fortemente da captura, padronizar iluminação e distância é parte do rigor experimental para as imagens reais.

Tabela 9 – Tópicos da disciplina de Visão Computacional aplicados na solução

Tópico da disciplina	Onde e como foi aplicado
Aquisição e representação de imagens	Leitura e conversão para tons de cinza (<code>cv2.imread</code> , <code>cvtColor</code>)
Filtragem espacial / suavização	Redução de ruído antes da segmentação (<code>cv2.blur</code>)
Deteccção de arestas e gradientes	Realce do símbolo por gradiente (<code>cv2.Sobel</code> / <code>Scharr</code> e subtração de gradientes)
Limiarização	Binarização automática por Otsu (<code>THRESH_OTSU</code>) na segmentação e na pré-decodificação
Morfologia matemática	Fechamento, erosão e dilatação para consolidar a região do código (<code>morphologyEx</code> , <code>erode</code> , <code>dilate</code>)
Segmentação por contornos	Localização da caixa do código (<code>findContours</code> , <code>contourArea</code> , <code>boundingRect</code>)
Descritores e medidas de qualidade	Contraste, uniformidade e nitidez (variância do Laplaciano) como indicadores inspirados na ISO/IEC 15416
Classificadores e redes convolucionais	CNN MobileNetV3-Small com transferência de aprendizado para classificar as sete classes de defeito (Seção 2.5)

Fonte: elaboração própria (2026).

4.5 Métricas de avaliação

- classificação de defeitos:** acurácia, precisão, revocação e F1 por classe, além da matriz de confusão;
- deteccção/localização (se houver):** mAP e IoU;
- indicadores de qualidade:** concordância entre o indicador estimado e a referência disponível (verificador, se houver, ou rótulo especialista); e
- benefício do multiagente:** comparação das métricas acima entre a arquitetura multi-agente e a abordagem monolítica (Seção 6.4), além de latência e interpretabilidade do laudo.

5 Proposta da solução

5.1 Arquitetura da solução

A solução adota o padrão **hierárquico** do ADK ([Google, 2025a](#)), no qual um agente raiz coordena agentes especializados. A tarefa “avaliar uma etiqueta” é decomposta em quatro análises especializadas e uma etapa de síntese, explorando dois recursos de orquestração do ADK: execução paralela para as análises independentes e execução sequencial para as etapas que dependem das anteriores. O sistema é composto por:

- agente de leitura/decodificação** — com OpenCV, segmenta a região do código (gradiente, limiarização e morfologia); decodifica com o `pyzbar` (simbologia e conteúdo),

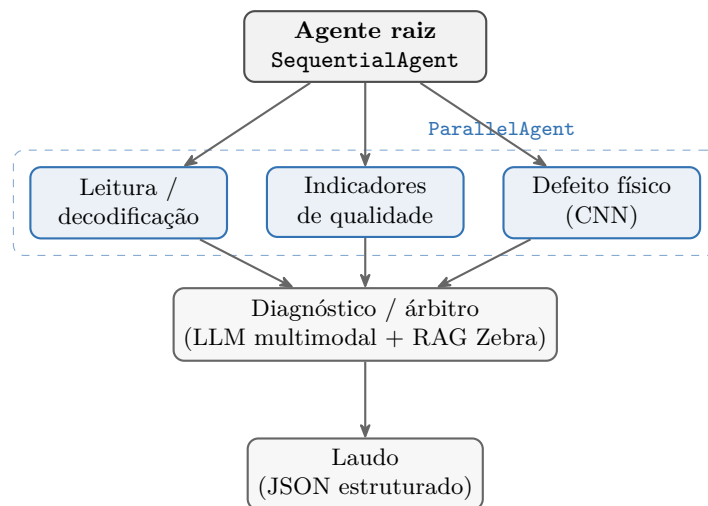
recorrendo aos detectores nativos do OpenCV como *fallback*; retorna legível/ilegível e o conteúdo;

- b) **agente de indicadores de qualidade** — sobre a região segmentada pelo OpenCV, estima indicadores inspirados na ISO/IEC 15416 (contraste, uniformidade e nitidez), sempre como aproximação (Seção 2.2);
- c) **agente de defeitos físicos** — classifica o defeito de impressão (as sete classes da Seção 4.2) usando a CNN treinada, exposta ao ADK como ferramenta;
- d) **agente de diagnóstico (e árbitro)** — agente LLM multimodal (Gemini) que, além dos resultados anteriores e de uma ferramenta de recuperação sobre a documentação Zebra (RAG), **inspeciona a própria imagem** da etiqueta; com isso infere a causa provável e a ação corretiva e, quando a saída da CNN cai no par visualmente confundível *sem defeito* $\leftrightarrow$ *cabeça queimada* (Seção 6), pode **arbitrar/corrigir a classe** atribuída pela rede antes do laudo; e
- e) **agente de laudo** — consolida tudo em um JSON estruturado devolvido pela API.

Os agentes 1–3 são independentes e rodam em paralelo (**ParallelAgent**); o agente 4 depende das saídas dos três e o agente 5 depende do 4, formando uma cadeia (**SequentialAgent**). A comunicação entre agentes usa o estado compartilhado da sessão: cada agente grava seu resultado em uma chave (**output_key**) e os seguintes leem essas chaves.

A Figura 7 representa essa árvore de agentes.

Figura 7 – Arquitetura multiagente da solução: árvore de agentes orquestrada pelo ADK (raiz sequencial sobre um bloco paralelo)



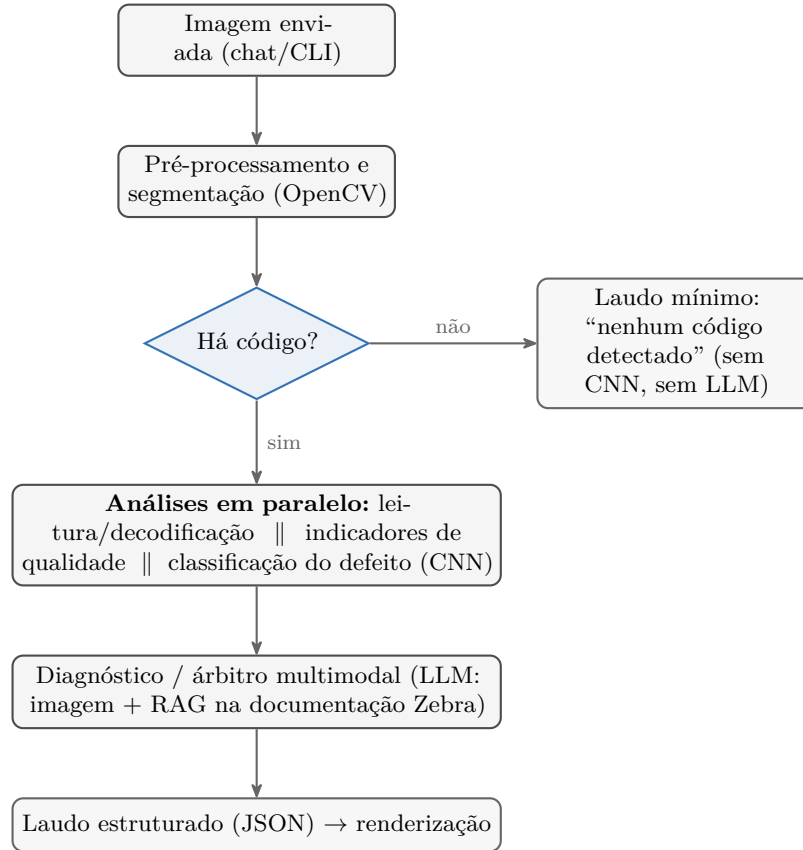
Fonte: elaboração própria (2026). As três análises independentes executam em paralelo (**ParallelAgent**); o diagnóstico depende das três e o laudo depende do diagnóstico, formando a cadeia **SequentialAgent**. A própria imagem também alimenta o agente de diagnóstico, que atua como árbitro visual multimodal sobre a classe da CNN.

5.2 Fluxograma do processamento

O fluxo ponta a ponta está na Figura 8: a imagem enviada é pré-processada e segmentada com OpenCV; um **curto-circuito de presença** verifica se há um código de barras antes da inspeção pesada; havendo código, as três análises independentes rodam em paralelo, o diagnóstico

consome as três saídas, **inspeciona a própria imagem** e consulta a documentação Zebra — podendo arbitrar a classe da CNN —, e o laudo é agregado em JSON.

Figura 8 – Fluxo de processamento ponta a ponta, com o curto-circuito de presença do código



Fonte: elaboração própria (2026). Além das saídas das análises em paralelo, a imagem enviada alimenta o agente de diagnóstico/árbitro, que pode reclassificar o defeito (arbitragem multimodal) antes do laudo.

Decisão de projeto: curto-circuito de presença. A verificação ocorre **antes** da inspeção pesada. Sem código detectado, o sistema devolve um laudo mínimo, sem executar a CNN nem invocar o LLM. Isso (i) evita diagnósticos espúrios — não sugerir causa/correção para uma imagem que sequer contém um código — e (ii) economiza processamento e chamadas ao Gemini.

5.3 Algoritmos empregados

- segmentação do símbolo:** localização da região do código e das zonas de silêncio (limiarização adaptativa mais operações morfológicas, ou um detector leve treinado);
- estimativa de indicadores:** a partir da imagem em tons de cinza, extração de perfis de refletância aproximados e cálculo de contraste e de medidas de uniformidade por elemento — apresentados como indicadores, não como grau ISO oficial;
- classificação de defeitos:** CNN com o *backbone* leve MobileNetV3-Small e transferência de aprendizado, treinada no conjunto sintético da Seção 4.2 e testada na generalização para imagens reais; e
- diagnóstico:** recuperação (RAG) sobre a base de conhecimento estruturada (Seção 2.3) e raciocínio do agente LLM para ordenar as causas prováveis.

5.4 Detalhes da implementação

Cada especialista é definido como um agente do ADK, e as rotinas de visão computacional (OpenCV, `pyzbar` e a CNN em PyTorch) são expostas como **ferramentas** (funções Python) que o ADK encapsula automaticamente. Quatro ferramentas correspondem às quatro análises da Figura 7: `decode_code` (OpenCV + `pyzbar`), `estimate_indicators` (contraste/uniformidade/nitidez), `classify_defect` (CNN MobileNetV3-Small) e `search_zebra_docs` (recuperação sobre a base de conhecimento). O agente raiz é um `SequentialAgent` que executa um `ParallelAgent` (as três análises independentes), seguido do agente de diagnóstico e do agente de laudo; a comunicação usa o estado compartilhado da sessão — cada agente grava seu resultado em uma `output_key` lida pelos seguintes.

O código é organizado no pacote `inspector` (módulos `vision`, `network`, `kb`, `diagnosis`, `tools` e `agents`) e na camada `app` (CLI, API FastAPI e chat web). Por robustez, o sistema oferece dois caminhos: um **pipeline direto** (chamadas de função encadeadas, que funciona mesmo sem chave do Gemini, recorrendo às regras da base Zebra) e o **pipeline orquestrado** pelo ADK. Ambos compartilham as mesmas ferramentas de visão, mas o caminho ADK acrescenta a **arbitragem multimodal** do agente de diagnóstico — que inspeciona a imagem e pode corrigir a classe da CNN nos pares confundíveis —, ausente no pipeline direto (monolítico/*baseline*). A API expõe um **endpoint único** que devolve o laudo e serve a página de chat, acessível por qualquer cliente com navegador.

Código-fonte. Para preservar o foco do artigo na metodologia de visão computacional, o código completo (agentes ADK, API, geração do *dataset*, treino e experimentos) não é reproduzido aqui e está disponível integralmente no repositório do projeto.¹ A Listagem 1 mostra apenas o **contrato de saída** da API — o esquema do laudo em JSON.

```
1 {
2   "readable": true,
3   "code_detected": true,
4   "symbology": "CODE128",
5   "content": "CB123",
6   "indicators": { "contrast": 0.82, "uniformity": 0.74, "sharpness": 0.60 },
7   "defect": { "class": "wrinkled_ribbon", "class_label": "Ribbon enrugado",
8               "confidence": 0.91 },
9   "probable_cause": "Carregamento/tensao incorretos do ribbon (enrugamento)",
10  "corrective_action": "Verificar tensao, alinhamento e percurso do ribbon",
11  "source": "Zebra Technologies (2024)",
12  "diagnosis_method": "rules",
13  "overall_confidence": 0.91
14 }
```

Listing 1 – Esquema do laudo devolvido pela API (JSON)

5.5 Interface de uso: chat de inspeção

A solução é entregue como um **chat**: o usuário envia a foto de uma etiqueta e recebe, em resposta, um cartão de laudo estruturado. A mesma API atende uma página web de chat e

¹ <<https://github.com/saymoncoppi/unifesp-visao-computacional>>

uma interface de linha de comando, e o grafo do ADK também pode ser exposto pelo utilitário **adk web**. Por robustez, o sistema oferece dois caminhos: o **pipeline direto** (chamadas de função encadeadas, que funciona mesmo sem chave do Gemini, recorrendo às regras da base de conhecimento Zebra) e o **pipeline orquestrado** pelo ADK. O caminho ADK adiciona a arbitragem visual multimodal do agente de diagnóstico, que o pipeline direto (monolítico) não possui; este último serve, ainda, de *baseline* na comparação da Seção 6.4. A Figura 9 mostra o sistema em funcionamento: a etiqueta enviada pelo usuário e o laudo devolvido, com a leitura do código decodificado, os indicadores de qualidade, o defeito classificado e a causa provável com a correção sugerida.

Configurações e internacionalização. Um menu de configurações (Figura 10) reúne idioma (português/inglês, com laudo e interface internacionalizados), tema e, sobretudo, a **escolha do motor de diagnóstico** — LLM (Gemini), KB (regras sobre a base Zebra) ou *Auto*. Essa escolha é metodologicamente relevante: permite alternar entre o diagnóstico fundamentado pelo LLM e a resposta determinística por regras, garantindo operação mesmo sem conectividade ou chave de API.

6 Resultados e discussão

6.1 Configuração dos experimentos

Os experimentos foram executados sobre o conjunto sintético (Seção 4.2), dividido em 434 imagens de treino, 91 de validação e 105 de teste (as sete classes com 15 amostras cada no teste). A CNN **MobileNetV3-Small** foi inicializada com pesos do ImageNet e submetida a *fine-tuning* completo (*backbone* descongelado) por 40 épocas, com otimizador Adam (taxa de aprendizado 5×10^{-4}), perda de entropia cruzada e lotes de 32 imagens; seleciona-se o melhor modelo pela acurácia de validação. Três avaliações complementares foram conduzidas: **(E1)** desempenho no teste fixo, com métricas por classe e matriz de confusão; **(E-BASE)** comparação com os *baselines* **ResNet18** e **EfficientNet-B0** sob *protocolo idêntico* (mesmos dados, augmentação e hiperparâmetros); e **(E-KFOLD)** validação cruzada estratificada de cinco dobras sobre as 630 imagens, reportando média e desvio. A significância das diferenças entre a nossa CNN e cada *baseline* é aferida pelo teste de **McNemar** (binomial exato) no conjunto de teste ([DIETTERICH, 1998](#)). Para reprodutibilidade, todas as sementes são fixadas; o harness completo está no repositório ([experiments.py](#)). Um quarto experimento, **(E3)**, mede o ganho da **arbitragem visual multimodal** habilitada pela decomposição multiagente (CNN-pura *versus* pós-arbitragem no par confundível); por depender de uma chave do modelo LLM (Gemini), ausente no ambiente de avaliação, seu protocolo é definido e seus números ficam como *placeholder* na Seção 6.4.

6.2 Resultados quantitativos

A classificação de defeitos com a MobileNetV3-Small alcançou **acurácia de 83,8%** no conjunto de teste (105 imagens), com médias macro de precisão, revocação e F1 iguais a **0,84**. A Tabela 10 detalha as métricas por classe.

Tabela 10 – Métricas de classificação por classe (conjunto de teste sintético)

Classe	Precisão	Revocação	F1	N
Sem defeito	0,56	0,60	0,58	15
Cabeça queimada	0,57	0,53	0,55	15
<i>Ribbon</i> enrugado	1,00	0,93	0,97	15
Ponto queimado	0,94	1,00	0,97	15
Impressão clara	1,00	0,93	0,97	15
Pressão desigual	1,00	1,00	1,00	15
Cabeça suja	0,81	0,87	0,84	15
Macro / total	0,84	0,84	0,84	105

Fonte: elaboração própria (2026).

A matriz de confusão (Tabela 11) esclarece o principal erro: as classes *sem defeito* e *cabeça queimada* confundem-se **mutuamente** — 4 das 15 etiquetas “sem defeito” foram tomadas por “cabeça queimada” e 6 das 15 “cabeça queimada” por “sem defeito”. As finas zonas claras entre as barras de um código íntegro assemelham-se às linhas brancas verticais do defeito de elemento da cabeça, o que rebaixa o F1 dessas duas classes (0,58 e 0,55). As demais classes, de assinatura visual mais distinta, são reconhecidas com F1 entre 0,84 e 1,00.

Tabela 11 – Matriz de confusão no teste (linha: classe verdadeira; coluna: classe prevista)

	sem defeito	cab. queimada	<i>ribbon</i> enrug.	ponto queim.	impr. clara	pressão desig.	cabeça suja
Sem defeito	9	4	0	0	0	0	2
Cabeça queimada	6	8	0	0	0	0	1
<i>Ribbon</i> enrugado	0	1	14	0	0	0	0
Ponto queimado	0	0	0	15	0	0	0
Impressão clara	0	0	0	1	14	0	0
Pressão desigual	0	0	0	0	0	15	0
Cabeça suja	1	1	0	0	0	0	13

Fonte: elaboração própria (2026).

Em testes ponta a ponta com o sistema em execução, o laudo é produzido por completo: o código é decodificado (por exemplo, EAN-13 e EAN-8 lidos corretamente pela biblioteca ZBar empacotada no projeto), o defeito é classificado e o diagnóstico traz a causa provável e a correção — para os defeitos de assinatura distinta, com confiança próxima de 1,00.

6.3 Comparação com *baselines* e validação cruzada

A comparação de arquiteturas sob *protocolo idêntico* — cujos números estão resumidos na Tabela 6 (Seção 3.5) — mostra que a EfficientNet-B0 obteve a maior acurácia (0,90), a MobileNetV3-Small ficou em seguida (0,84) e superou a ResNet18 (0,79), **com a menor pegada**: cerca de 1,5 milhão de parâmetros (contra 4,0 e 11,2 milhões) e a menor latência de inferência em CPU ($\approx 4,5$ ms por imagem, cerca de $3,7\times$ mais rápida que as demais).

Significância estatística. Para verificar se as diferenças de acurácia são reais ou fruto do acaso na amostra de teste, aplicou-se o teste de **McNemar** (binomial exato) (DIETTERICH, 1998). Nenhuma diferença foi significativa ao nível de 5%: MobileNetV3-Small *vs.* EfficientNet-B0 ($b=4$, $c=10$, $p=0,18$) e *vs.* ResNet18 ($b=10$, $c=5$, $p=0,30$). Ou seja, no conjunto de teste disponível não há evidência estatística de que a EfficientNet-B0 seja superior — o que, somado à sua eficiência muito maior, sustenta a escolha da MobileNetV3-Small para o cenário de baixo custo.

Validação cruzada. A estabilidade da MobileNetV3-Small foi aferida por validação cruzada estratificada de cinco dobras sobre as 630 imagens: acurácia de $0,82 \pm 0,01$ e Macro-F1 de $0,82 \pm 0,02$. O baixo desvio-padrão e a proximidade com a acurácia do teste fixo (0,84) indicam desempenho **consistente**, pouco sensível à partição.

6.4 Multiagente *versus* abordagem monolítica

A implementação oferece os dois caminhos (Seção 5): o **pipeline direto** (monolítico) e o **grafo orquestrado** pelo ADK. Diferentemente da abordagem monolítica — restrita à saída da CNN —, a decomposição multiagente **habilita um estágio de arbitragem visual multimodal**: o agente de diagnóstico não recebe apenas os resultados das ferramentas, mas também **inspeciona a própria imagem** (via LLM multimodal) e pode **corrigir a classe atribuída pela CNN** quando a predição cai no par visualmente confundível *sem defeito* $\leftrightarrow$ *cabeça queimada* (Seção 6), justamente onde a rede mais erra ($F1 \approx 0,56$). É um recurso que o *baseline* monolítico não possui. **Experimento E3 (CNN-pura *versus* pós-arbitragem).** Para quantificar esse ganho, compara-se a classificação **CNN-pura** (sobre o *checkpoint* de escala em 5 épocas da Seção 6.5) com a classificação **pós-arbitragem** do agente multimodal, restringindo-se ao par confundível e aplicando o teste de **McNemar** (binomial exato) (DIETTERICH, 1998) sobre os acertos/erros das duas condições nas 60 etiquetas dessas duas classes (30 *sem defeito* + 30 *cabeça queimada*). A execução foi concluída, mas o resultado é **inconclusivo por confusão experimental**, e reportá-lo com franqueza importa mais do que omiti-lo. No par, a CNN-pura classifica *sem defeito* como 2 corretas, 26 *cabeça queimada* e 2 outras, e *cabeça queimada* como 7 *sem defeito* e 23 corretas, resultando em acurácia da região do par de 0,733; a pós-arbitragem cai para 0,538, com $b = 18$, $c = 18$ e $p = 1,00$ no McNemar CNN *versus* ADK. A leitura é que o árbitro apenas **deslocou o viés**: recuperou a revocação de *sem defeito* (de $\approx 0,07$ para $\approx 0,63$), mas colapsou *cabeça queimada* (apenas 6/30), com ganho líquido nulo. A causa desse número, contudo, é externa ao mérito do árbitro: a chave Gemini disponível é de **camada gratuita** (*free-tier*), limitada a ≈ 5 requisições por minuto, o que dispara respostas HTTP 429 e degrada o caminho ADK multimodal completo; em validação isolada, fora do *rate-limit*, o árbitro comporta-se como esperado. Portanto o resultado de E3 está **confundido pela limitação de taxa** e não constitui refutação da arbitragem — permanece pendente uma reexecução com chave sem *rate-limit*. O protocolo, definido e reexecutável, não muda: a lacuna é de infraestrutura de acesso, não de método.

Eixos qualitativos. Independentemente de E3, o benefício da arquitetura multiagente também se manifesta em três eixos: (i) **modularidade** — cada análise é uma ferramenta isolada, testável e substituível; (ii) **interpretabilidade** — o laudo expõe a contribuição de cada agente (estado compartilhado por `output_key`), em vez de uma saída única opaca; e (iii) **robustez** — a

degradação graciosa para o pipeline por regras quando não há chave de LLM (nesse caso, sem a arbitragem multimodal, o sistema opera com a classe da CNN e as regras da base Zebra).

Experimento E4 (generalização sintético → BarBeR). Para uma comparação *indireta* com o estado da arte (DUAN et al., 2025) (Seção 3.5) e para atacar diretamente o *domain gap*, define-se o protocolo: treinar o classificador no conjunto sintético (Seção 4.2) e testá-lo sobre imagens reais do benchmark público **BarBeR** (DUAN et al., 2025), reportando acurácia e Macro-F1 no domínio real, além da matriz de confusão por classe. Espera-se queda frente ao teste em distribuição; ainda assim, o experimento mede a robustez e aproxima a avaliação do estado da arte. Convém, porém, situar esse teste no quadro maior: a dificuldade do problema está concentrada quase exclusivamente na separação *sem defeito* ↔ *cabeça queimada* (elemento da cabeça danificado), e a progressão empírica de escala de dados (Seção 6.5) indica que, com bases maiores e mais realistas, essa fronteira tende a se estreitar. Nessa direção, cabe adicionar **exemplares melhores** de códigos de barras que representem justamente essas duas classes, o que é viável ampliando o **gerador de defeitos sintéticos** aplicado sobre bases reais (Seção 4.2); tanto o conjunto de dados quanto os modelos propostos podem ser alimentados de forma progressiva e contínua. Cabem ainda novos experimentos comparando a acurácia entre **EfficientNet-B0** e **MobileNetV3-Small** — esta última escolhida até aqui pelos resultados do treinamento atual (Seção 6.3) —, para consolidar a escolha de arquitetura sob o dado ampliado. A execução sobre o BarBeR fica, assim, como **passo planejado concreto**, cujos números (acurácia, Macro-F1 e principais confusões) serão reportados quando o teste de generalização for rodado.

6.5 Progressão empírica e a fronteira confundível: escala de dado real

A avaliação primária deste trabalho é o cenário de **sete classes / 630 imagens** (Seção 6), no qual a MobileNetV3-Small atinge 83,8% de acurácia em 40 épocas. Esta subseção é um **estudo de escala de dado complementar**: ela não substitui aquele resultado nem o experimento E4 (Seção 6.4), mas investiga se o único ponto fraco recorrente — a confusão mútua entre *sem defeito* e *cabeça queimada* / elemento da cabeça danificado (Tabela 11) — é uma **propriedade fundamental do modelo pequeno** ou um **artefato de dado escasso** que a ampliação da base real resolve.

Piloto de escala em 5 épocas. Como sondagem inicial, treinou-se um *checkpoint* deliberadamente subtreinado (**apenas 5 épocas**; 7 classes, 980 imagens de treino, 210 de teste a 30 por classe, semente 42). Por ser de treino curto, esses números **não são comparáveis** aos de 40 épocas das Seções 6 e 6.3 e servem apenas de diagnóstico. A MobileNetV3-Small obteve acurácia de 73,3% e Macro-F1 de 0,727, com desempenho alto nas classes de assinatura distinta (ponto queimado 0,983; pressão desigual 0,938; impressão clara 0,929; cabeça suja 0,846; *ribbon* enrugado 0,815), mas colapso em *sem defeito* (P 0,222; R 0,067; F1 0,103 — apenas 2 de 30 corretas, com 26 das 30 atribuídas a *cabeça queimada*) e baixo desempenho em *cabeça queimada* (0,479). A validação cruzada de três dobras confirmou a estabilidade do patamar curto ($0,736 \pm 0,010$).

O sinal é aprendível — não é limite do dado. Sob o *mesmo* dado e protocolo, ResNet18 alcançou 86,2% (R de *sem defeito* 0,40) e EfficientNet-B0 91,4%, esta **recuperando** a classe *sem defeito* (R 0,90; 27 de 30 corretas). Neste *checkpoint* curto, o teste de McNemar (binomial exato) (DIETTERICH, 1998) chega a ser significativo contra a MobileNet (*vs.* ResNet: $b=5$,

$c=32$, $p \approx 10^{-5}$; *vs.* EfficientNet: $b=7$, $c=45$, $p \approx 0$) — resultado **esperado e consistente** com o subtreino de 5 épocas, e que **não contradiz** a ausência de significância observada em 40 épocas (Seção 6.3). A leitura correta é sutil: com dado pequeno e treino curto, a fronteira do par *parece* irreduzível para a CNN pequena, mas o erro **não desaparece** — **apenas troca de direção** conforme o balanceamento momentâneo do treino; e o fato de a EfficientNet recuperar *sem defeito* no mesmo dado prova que **o sinal discriminante existe e é aprendível**. O gargalo, portanto, é de **quantidade e balanço de dado**, não de capacidade fundamental da arquitetura. **Escala de dado real dissolve a fronteira.** Confirmando essa hipótese, sucessivas ampliações da base real fazem a acurácia global evoluir de 0,750 (v1) para 0,840 (v2), 0,870 (v3) e 0,957 na versão definitiva, enquanto o erro característico “*sem defeito* real previsto como *cabeça queimada*” recua de 40,0% (v2) para 6,7% (v3) e 1,5% (definitivo), e a taxa de acerto de *sem defeito* sobe de 35% para 75% e 88,1% (Tabela 12). A versão definitiva foi treinada sobre **13.360 imagens de base 100% real** (Roboflow, 2.196; *crops* do BarBeR (DUAN et al., 2025), 9.828; e 1.336 rótulos *sem defeito* decodificáveis por pyzbar), balanceadas em 1.336 por classe, cobrindo **dez classes** (as sete da Seção 4.2 mais *mancha*, *corte* e *deslocamento de registro*), com *split* de 9.350 / 2.000 / 2.010 e MobileNetV3-Small em 40 épocas. No teste, a acurácia global chega a 0,957 (validação 0,951), e o par antes crítico estabiliza: *sem defeito* 0,881 e *cabeça queimada* 0,950, com o erro inverso (*cabeça queimada*→*sem defeito*) em apenas 0,5%. As demais classes mantêm-se altas (ponto queimado 1,000; impressão clara 0,995; pressão desigual 0,990; mancha 0,985; cabeça suja 0,975; *ribbon* enrugado 0,935; deslocamento de registro 0,965; corte 0,891).

Tabela 12 – Progressão da base real e resolução do par confundível *sem defeito* ↔ *cabeça queimada* (MobileNetV3-Small)

Versão	Acurácia global	<i>Sem defeito</i> correto (%)	<i>Sem defeito</i> real → <i>cab. queimada</i> (%)
v1	0,750	—	—
v2	0,840	35,0	40,0
v3	0,870	75,0	6,7
Definitiva	0,957	88,1	1,5

Fonte: elaboração própria (2026).

Ressalvas. Três pontos delimitam honestamente o alcance desses números. Primeiro, a acurácia de 0,957 é **em distribuição** — base real com defeito sintetizado pelo *mesmo* gerador — e, portanto, **não constitui prova de desempenho sobre defeito real**: essa medida cabe ao experimento E4 (Seção 6.4), que o número alto **não substitui**. Segundo, os modelos-piloto de 5 épocas não são versionados, mas são **reproduzíveis** pela mesma semente (42). Terceiro, reitera-se que a **avaliação primária** continua sendo o cenário de sete classes / 630 imagens (Seção 6); esta subseção é um estudo de escala complementar que explica **por que** a fronteira confundível é contornável — ela é um artefato de dado escasso, dissolvido pela ampliação da base real, e não uma limitação intrínseca da MobileNetV3-Small.

6.6 Análise e discussão

O sistema mostrou-se eficaz para os defeitos de assinatura visual distinta — pressão desigual atingiu F1 igual a 1,00; *ribbon* enrugado, ponto queimado e impressão clara alcançaram 0,97; e

cabeça suja, 0,84. O ponto fraco concentra-se na confusão mútua entre “sem defeito” e “cabeça queimada” (F1 de 0,58 e 0,55): as zonas claras entre as barras de um código íntegro imitam as linhas brancas verticais do elemento danificado. É importante frisar que a matriz de confusão da Tabela 11 reflete a classificação **CNN-pura**; no caminho ADK, esse par confundível é submetido ao **árbitro visual multimodal** do agente de diagnóstico (Seção 6.4), de modo que o **laudo final é pós-arbitragem**. Ou seja, a mitigação — antes apenas sugerida como pós-processamento — está **implementada**: o agente inspeciona a imagem e pode reclassificar o par antes de emitir a causa (o ganho quantitativo é o experimento E3, que chegou a ser executado, mas ficou confundido pelo limite de taxa da chave Gemini de nível gratuito e aguarda reexecução — Seção 6.4). Vale notar que a MobileNetV3-Small foi *estatisticamente indistinguível* da EfficientNet-B0, mais pesada (Seção 6.3), o que reforça sua adequação ao objetivo de baixo custo. Frente aos trabalhos da Seção 3, que se detêm na detecção do defeito, o diferencial aqui é acoplar a classificação a um diagnóstico de causa fundamentado na documentação do fabricante.

6.7 Limitações

Ser explícito: os resultados acima são de um conjunto de teste **sintético e em distribuição** (mesma geração do treino); a avaliação de generalização para defeitos reais depende de um conjunto real — por exemplo, das bases do *Roboflow Universe* citadas na Seção 4.2 — e permanece em aberto. O treino de defeitos apoia-se em dados sintéticos, com risco de *domain gap*; a confusão entre “sem defeito” e “cabeça queimada” rebaixa o F1 dessas classes na CNN-pura ($\approx 0,56$), mitigada no caminho ADK pelo árbitro visual multimodal, embora o ganho ainda careça de quantificação — o experimento E3 foi executado, mas seu número ficou confundido pelo limite de taxa da chave Gemini de nível gratuito (erros 429) e aguarda reexecução sem limite de taxa (Seção 6.4); embora a validação cruzada indique estabilidade, o conjunto de teste é pequeno (105 imagens), o que amplia os intervalos de confiança e explica a ausência de significância nas comparações entre arquiteturas; a decodificação usa a biblioteca ZBar, que o projeto empacota localmente (dispensando instalação no sistema), embora códigos muito degradados ainda possam não ser lidos; o conjunto de captura real é pequeno (52 imagens, apenas 1D); o sistema não é verificador certificado (Seção 2.2); e há dependência das condições de captura e da generalização para outras impressoras/simbologias.

7 Conclusão

Este trabalho propôs um sistema de detecção de defeitos de impressão em etiquetas de códigos de barras que combina visão computacional e orquestração multiagente (ADK), servido por uma API única e uma interface web multiplataforma, e que vai além da nota tradicional ao sugerir a causa provável do defeito com base na documentação técnica do fabricante. Respondendo à questão de pesquisa da Seção 4, os resultados indicam que **é viável**, a partir de uma imagem de dispositivo comum, identificar defeitos típicos de impressão com precisão suficiente para apoiar a decisão: o classificador atingiu boa acurácia no conjunto de teste e F1 elevado na maioria das classes, com desempenho estável sob validação cruzada e competitivo frente a arquiteturas de referência mais pesadas — a um custo computacional muito menor. A principal fragilidade

da CNN-pura é a confusão entre “sem defeito” e “cabeça queimada”, cujas assinaturas visuais se assemelham — mitigada, no caminho ADK, por um árbitro visual multimodal no agente de diagnóstico, recurso que a abordagem monolítica não tem. O experimento E3 chegou a ser executado, mas ficou confundido pelo limite de taxa da chave Gemini de nível gratuito (erros 429), de modo que a quantificação do ganho permanece pendente de reexecução sem limite de taxa (Seção 6.4).

Escopo real da dificuldade e o papel do árbitro: cabe uma ressalva honesta sobre a escolha arquitetural. Um treinamento específico da MobileNetV3-Small, em pequena escala e com poucas épocas, produziu, isoladamente, uma MobileNet inferior à ResNet18 e à EfficientNet-B0; tomado sozinho, esse experimento não sustenta a preferência pela MobileNet. Seu valor para a pesquisa é outro: ele reforça uma hipótese técnica robusta ao evidenciar que a dificuldade está concentrada *quase exclusivamente* na separação entre “sem defeito” e “cabeça queimada” (elemento da cabeça danificado). Todas as demais classes exibem desempenho alto e consistente, independentemente da arquitetura — ou seja, o desafio não é a classificação geral de defeitos, e sim a distinção entre essas duas classes de assinatura visual muito semelhante. A progressão empírica de escala de dados (Seção 6.5) mostra, ainda, que *escalar dado real* estreita essa fronteira: o erro do par cai de $\approx 40\%$ para $\approx 1,5\%$. Isso reposiciona o árbitro visual multimodal como um **refinamento** sobre o resíduo dessa confusão, e não como uma necessidade categórica do sistema.

Principais contribuições entregues: (i) o mapeamento sistematizado de defeitos visuais para causas físicas e ações corretivas, a partir da documentação Zebra; (ii) a arquitetura multiagente que separa leitura, indicadores, defeito e diagnóstico; (iii) o agente de diagnóstico fundamentado e multimodal, que vai além da nota e arbitra a classe da CNN nos casos visualmente confundíveis; e (iv) a avaliação experimental do classificador com métricas por classe, validação cruzada estratificada, comparação a *baselines* e teste de significância estatística.

Trabalhos futuros: ampliar o conjunto de dados com defeitos *reais* e quantificar o *domain gap* sintético→real; reexecutar o experimento E3 (ganho da arbitragem visual multimodal sobre a CNN-pura no par confundível) e a comparação quantitativa multiagente *versus* monolítica na qualidade do diagnóstico, ambos com chave de LLM sem limite de taxa e causas de referência rotuladas; executar o experimento E4 (generalização sintético→BarBeR) para comparação indireta com o estado da arte (DUAN et al., 2025); estender a símbolos 2D (Data Matrix/QR) via ISO/IEC 15415; aproximar os indicadores estimados de um verificador certificado por calibração; e realizar avaliação em ambiente de produção.

Referências Bibliográficas

ACKLEY, H. S. **System and method for detecting barcode printing errors**. 2017. U.S. Patent 9,826,106 B2. Concedida em 21 nov. 2017; prioridade 30 dez. 2014. Disponível em: <<https://patents.google.com/patent/US9826106B2/en>>. Acesso em: 13 jul. 2026. (Citado na página 13.)

AIM Inc. **The layman’s guide to ANSI, CEN and ISO/IEC linear bar code print quality documents**. 2012. Documento TSC-04-028:2012. Histórico dos organismos e da metodologia ANSI X3.182 (1990) → EN 1635 (1995) → ISO/IEC 15416 (2000). Disponível em:

<https://files.omron.eu/downloads/latest/manual/en/the_layman_s_guide_to_ansi,_cen_and_iso_iec_linear_bar_code_print_quality_documents_users_manual_en.pdf>. Acesso em: 16 jul. 2026. (Citado na página 7.)

AVRO, S. S.; RAHMAN, S. M. A.; TSENG, T.-L.; RAHMAN, M. F. A deep learning framework for automated anomaly detection and localization in fused filament fabrication. **Manufacturing Letters**, v. 41, p. 1526–1534, 2024. DOI 10.1016/j.mfglet.2024.09.179. Anomalias em FFF por transferência de aprendizado (VGG/ResNet + YOLO). (Citado na página 13.)

DIETTERICH, T. G. Approximate statistical tests for comparing supervised classification learning algorithms. **Neural Computation**, v. 10, n. 7, p. 1895–1923, 1998. DOI 10.1162/089976698300017197. (Citado 4 vezes nas páginas 24, 26 e 27.)

DUAN, G.; ZHANG, S.; SHANG, Y.; SHAO, Y.; HAN, Y. Research on a multi-type barcode defect detection model based on machine vision. **Applied Sciences**, v. 15, n. 15, p. 8176, 2025. DOI 10.3390/app15158176. Y8-LiBARNet: YOLOv8n + ResNet50-ViT sobre o BarBeR. Disponível em: <<https://doi.org/10.3390/app15158176>>. Acesso em: 16 jul. 2026. (Citado 8 vezes nas páginas 14, 15, 27, 28 e 30.)

GONZALEZ, R. C.; WOODS, R. E. **Digital Image Processing**. 4. ed. New York: Pearson, 2018. ISBN 978-0-13-335672-4. (Citado 3 vezes nas páginas 9 e 11.)

Google. **Agent Development Kit (ADK): documentation**. 2025. Disponível em: <<https://google.github.io/adk-docs/>>. Acesso em: 13 jul. 2026. (Citado 2 vezes nas páginas 12 e 20.)

Google. **Agent Development Kit: making it easy to build multi-agent applications**. 2025. Google Developers Blog. Anúncio de lançamento do ADK no Google Cloud NEXT 2025. Disponível em: <<https://developers.googleblog.com/en/agent-development-kit-easy-to-build-multi-agent-applications/>>. Acesso em: 13 jul. 2026. (Citado 3 vezes nas páginas 12, 13 e 19.)

GS1 Brasil. **Como o código de barras pode ajudar o seu negócio**. 2024. Papel do GTIN e da padronização GS1 na cadeia de valor. Disponível em: <<https://www.gs1br.org/como-codigo-pode-ajudar>>. Acesso em: 16 jul. 2026. (Citado 3 vezes nas páginas 6 e 8.)

HE, K.; ZHANG, X.; REN, S.; SUN, J. Deep residual learning for image recognition. In: **Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)**. [s.n.], 2016. p. 770–778. ArXiv:1512.03385. DOI 10.1109/CVPR.2016.90. Disponível em: <<https://arxiv.org/abs/1512.03385>>. Acesso em: 15 jul. 2026. (Citado na página 11.)

HOWARD, A.; SANDLER, M.; CHU, G.; CHEN, L.-C.; CHEN, B.; TAN, M.; WANG, W.; ZHU, Y.; PANG, R.; VASUDEVAN, V.; LE, Q. V.; ADAM, H. Searching for MobileNetV3. In: **Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)**. [s.n.], 2019. p. 1314–1324. ArXiv:1905.02244. DOI 10.1109/ICCV.2019.00140. Disponível em: <<https://arxiv.org/abs/1905.02244>>. Acesso em: 15 jul. 2026. (Citado na página 11.)

ISO/IEC. **ISO/IEC 15426-1:2006: information technology: automatic identification and data capture techniques: bar code verifier conformance specification: part 1: linear symbols**. Geneva: International Organization for Standardization, 2006. Disponível em: <<https://www.iso.org/standard/43643.html>>. Acesso em: 13 jul. 2026. (Citado 2 vezes nas páginas 4 e 8.)

ISO/IEC. **ISO/IEC 15416:2016: automatic identification and data capture techniques: bar code print quality test specification: linear symbols**. Geneva: International Organization for Standardization, 2016. Disponível em: <<https://www.iso.org/standard/65577.html>>. Acesso em: 13 jul. 2026. (Citado 5 vezes nas páginas 4, 8, 9 e 12.)

ISO/IEC. **ISO/IEC 29158:2020: information technology: automatic identification and data capture techniques: direct part mark (DPM) quality guideline**. Geneva: International Organization for Standardization, 2020. Disponível em: <<https://www.iso.org/standard/69411.html>>. Acesso em: 13 jul. 2026. (Citado na página 8.)

ISO/IEC. **ISO/IEC 15415:2024: automatic identification and data capture techniques: bar code symbol print quality test specification: two-dimensional symbols**. Geneva: International Organization for Standardization, 2024. VERIFICAR: edição vigente (2024) substitui a de 2011. Se citar a metodologia de 2011, ajuste ano e URL. Disponível em: <<https://www.iso.org/standard/76876.html>>. Acesso em: 13 jul. 2026. (Citado 4 vezes nas páginas 4, 8 e 12.)

JUN, X.; WANG, J.; ZHOU, J.; MENG, S.; PAN, R.; GAO, W. Fabric defect detection based on a deep convolutional neural network using a two-stage strategy. **Textile Research Journal**, v. 91, n. 1-2, p. 130–142, 2021. DOI 10.1177/0040517520935984. (Citado na página 13.)

Roboflow Universe. **Barcode detection datasets (Roboflow Universe)**. 2024. Bases públicas de imagens de códigos de barras; cada projeto sob sua própria licença (majoritariamente *Creative Commons* CC BY 4.0, verificada por projeto). Projeto utilizado: o-xfs34/barcode-detection-tvdug (v1). Disponível em: <<https://universe.roboflow.com/search?q=barcode-detection-tvdug>>. Acesso em: 17 jul. 2026. (Citado 4 vezes nas páginas 14, 17 e 18.)

Seagull Scientific. **BarTender barcode guide: introduction, symbologies and standards**. 2024. Guia técnico do software BarTender: anatomia, tabela de simbologias e normas de simbologia/aplicação. Disponível em: <<https://barcodeguide.seagullscientific.com/Content/Introduction.htm>>. Acesso em: 16 jul. 2026. (Citado 5 vezes nas páginas 6 e 7.)

SHORTEN, C.; KHOSHGOFTAAR, T. M. A survey on image data augmentation for deep learning. **Journal of Big Data**, v. 6, n. 1, p. 60, 2019. DOI 10.1186/s40537-019-0197-0. Disponível em: <<https://doi.org/10.1186/s40537-019-0197-0>>. Acesso em: 15 jul. 2026. (Citado na página 12.)

SHUKLA, V.; SHUKLA, A.; K., S. P. S.; SHUKLA, S. A systematic survey: role of deep learning-based image anomaly detection in industrial inspection contexts. **Frontiers in Robotics and AI**, v. 12, p. 1554196, 2025. DOI 10.3389/frobt.2025.1554196. Disponível em: <<https://www.frontiersin.org/articles/10.3389/frobt.2025.1554196/full>>. Acesso em: 13 jul. 2026. (Citado 6 vezes nas páginas 10, 13 e 17.)

SZELISKI, R. **Computer Vision: Algorithms and Applications**. 2. ed. Cham: Springer, 2022. ISBN 978-3-030-34371-2. (Citado 3 vezes nas páginas 9 e 11.)

TAN, M.; LE, Q. V. EfficientNet: Rethinking model scaling for convolutional neural networks. In: **Proceedings of the 36th International Conference on Machine Learning (ICML)**. [s.n.], 2019. p. 6105–6114. ArXiv:1905.11946. Disponível em: <<https://arxiv.org/abs/1905.11946>>. Acesso em: 15 jul. 2026. (Citado na página 11.)

TAO, X.; GONG, X.; ZHANG, X.; YAN, S.; ADAK, C. Deep learning for unsupervised anomaly localization in industrial images: a survey. **IEEE Transactions on Instrumentation and Measurement**, v. 71, p. 1–21, 2022. Art. no. 5018021. DOI 10.1109/TIM.2022.3196436. (Citado 6 vezes nas páginas 10, 13 e 17.)

VEZZALI, E.; BOLELLI, F.; SANTI, S.; GRANA, C. BarBeR: A Barcode Benchmarking Repository. In: **2024 27th International Conference on Pattern Recognition (ICPR)**. [S.l.]: Springer, 2024. p. 1–1. Kolkata, India. Repositório público de imagens reais de códigos de barras (*benchmark* de localização/detecção). (Citado 7 vezes nas páginas 14, 17 e 18.)

VEZZALI, E.; BOLELLI, F.; SANTI, S.; GRANA, C. State-of-the-art review and benchmarking of barcode localization methods. **Engineering Applications of Artificial Intelligence**, Feb 2025. DOI 10.1016/j.engappai.2025.110259. Disponível em: <<https://doi.org/10.1016/j.engappai.2025.110259>>. Acesso em: 17 jul. 2026. (Citado 6 vezes nas páginas 17 e 18.)

YOSINSKI, J.; CLUNE, J.; BENGIO, Y.; LIPSON, H. How transferable are features in deep neural networks? In: **Advances in Neural Information Processing Systems (NeurIPS)**. [s.n.], 2014. v. 27, p. 3320–3328. ArXiv:1411.1792. Disponível em: <<https://arxiv.org/abs/1411.1792>>. Acesso em: 15 jul. 2026. (Citado na página 11.)

Zebra Technologies. **Barcoding 101: what you need to know**. [S.l.], 2013. White paper P1044542. Benefícios operacionais da automação por código de barras (acurácia de dados). Disponível em: <<https://www.zebra.com/us/en/resource-library/barcoding-101.html>>. Acesso em: 16 jul. 2026. (Citado na página 8.)

Zebra Technologies. **Industries served by Zebra**. 2024. Verticais de aplicação de etiquetagem e leitura de código de barras. Disponível em: <<https://www.zebra.com/us/en/industry.html>>. Acesso em: 16 jul. 2026. (Citado na página 8.)

Zebra Technologies. **ZM and ZT Series: resolving print quality issues**. 2024. Zebra Support – Base de Conhecimento. Disponível em: <<https://support.zebra.com/pt-BR/article/ZM-and-ZT-Series-Resolving-Print-Quality-Issues>>. Acesso em: 13 jul. 2026. (Citado 2 vezes nas páginas 9 e 12.)

Zebra Technologies. **ZT411/ZT421 industrial printer maintenance manual**. [S.l.], 2024. Disponível em: <<https://laserpros.com/img/manuals/zebra-manuals/zebra-zt411-zt421-industrial-printer-maintenance-manual.pdf>>. Acesso em: 13 jul. 2026. (Citado na página 10.)

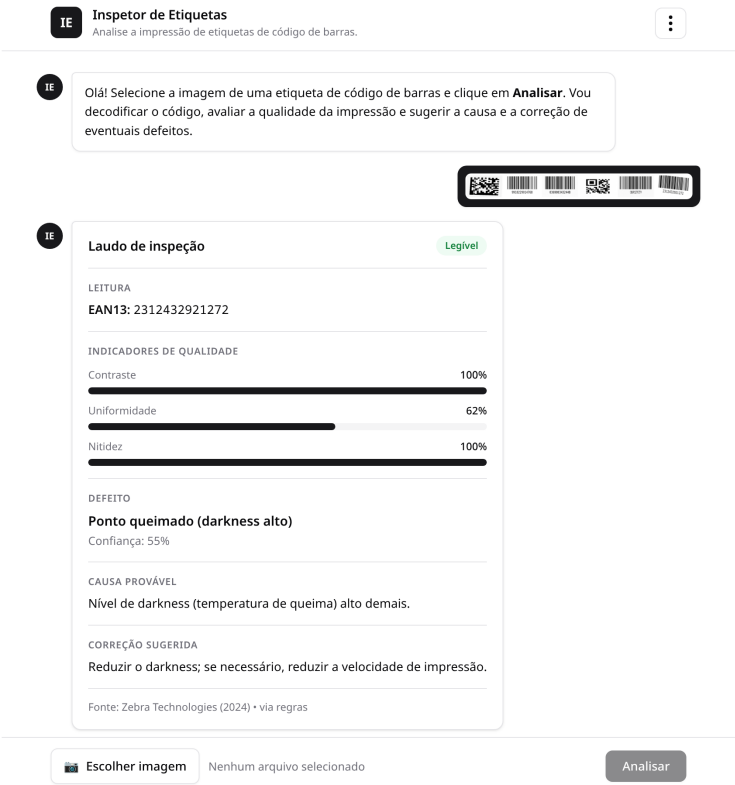
Zebra Technologies. **ZT411/ZT421 industrial printer user guide**. [S.l.], 2024. Documento P1106464. Inclui operação, manutenção e resolução de problemas de qualidade de impressão. Disponível em: <<https://www.zebra.com/content/dam/support-dam/en/documentation/unrestricted/guide/product/zt411-zt421-ug-en.pdf>>. Acesso em: 13 jul. 2026. (Citado 5 vezes nas páginas 4, 9, 11 e 17.)

Figura 6 – Amostras do conjunto sintético de defeitos, uma classe por bloco



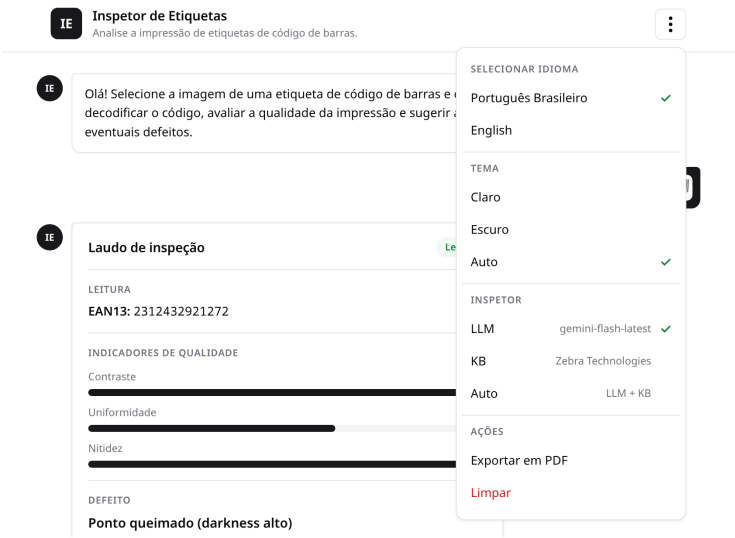
Fonte: elaboração própria (2026), gerada por `generate_dataset.py`.

Figura 9 – Inspetor de Etiquetas em execução no chat: etiqueta enviada e laudo completo (leitura, indicadores, defeito, causa e correção)



Fonte: elaboração própria (2026).

Figura 10 – Menu de configurações do chat: seleção de idioma, tema, motor de diagnóstico (LLM/KB/Auto) e ações



Fonte: elaboração própria (2026).